Build a Production SaaS Platform From Scratch

A rebuild-from-zero course in architecture, engineering practice, and the decisions behind them

Perezosoft Platform · generated 2026-07-08

Contents

Preface

What this book is

This is a course disguised as a book. Over ten parts and fifty lessons, you will build — by typing every hand-written line yourself — a production-grade, multi-tenant SaaS platform: custom JWT authentication with passwordless sign-in, OAuth, and MFA; tenant isolation enforced by the type system *and* by the database; billing with Stripe; reliable asynchronous work via a transactional outbox; role-based access control; file storage; GDPR export and erasure; a public API; outbound webhooks; an admin back-office; full observability — and at the end you will deploy it to a real host behind a real CI/CD pipeline that you also built.

The code is real. Every excerpt in this book is drawn from a working reference platform (Perezosoft Platform), not invented for the page. A coverage gate — a script, in the same spirit as the tests you'll write — maps every file of that platform to the lesson that builds it, so nothing is skipped and nothing appears out of thin air.

But the code is not the point. The point is the two habits that separate an architect from a very fast typist:

1. **Making invariants structural.** Not "remember to filter by tenant" but "a query *cannot* forget to filter by tenant." Not "please don't call this API in feature code" but "the build fails if you do." You will meet this move dozens of times — in query filters, interceptors, marker interfaces, architecture tests, CI gates, database policies — until reaching for it is a reflex.
1. **Recording decisions.** Every lesson ends with an *Architecture Decision* box: the fork that was faced, the option chosen, the options rejected, and the price paid. You will write your own decision log from lesson 0.1 onward, before any code exists. Six months from now, "why is it like this?" will have an answer.

Who this is for

You are comfortable in C# — classes, interfaces, async/await, LINQ hold no fear — and you can find your way around a terminal and git. You have probably built applications, perhaps professionally, but you want the *architecture* to stop feeling like folklore: why layers, why seams, why tests first, why all these patterns with strange names, and — above all — when *not* to use them.

You do not need prior experience with multi-tenancy, EF Core internals, Stripe, Docker, OAuth, or GitHub Actions. Each is taught when the platform needs it, motivated by a concrete problem you will feel before you solve it.

What this book is not

Honesty up front: this is a course in **systems design and engineering practice**, not computational problem-solving. You will find no algorithm puzzles and almost no performance engineering here — no profilers, and caching is deliberately deferred (a recorded decision you'll read about). If you want to get better at data structures, this is the

wrong book; if you want to get better at *building software that survives contact with production and with other people*, it is the right one.

The rhythm

Every lesson follows the same beat, and the beat is the curriculum:

1. **Motivate** — a concrete problem, ideally demonstrated by a failing or *leaking* test.
2. **Red** — write the test that pins the behavior you want. It fails.
3. **Green** — write the minimum code to pass it.
4. **Refactor & harden** — extract the seam; add the guardrail that makes the mistake impossible to repeat.

1. **Run it** — commands and expected output; every lesson ends with the app working.
2. **Architecture Decision** — the fork, the choice, the road not taken.
3. **Checkpoint** — a git tag; what you should now be able to do.

Test-driven development is not a chapter in this book; it is the loop you will live roughly fifty times, exactly as the reference platform was actually built.

How to use this book

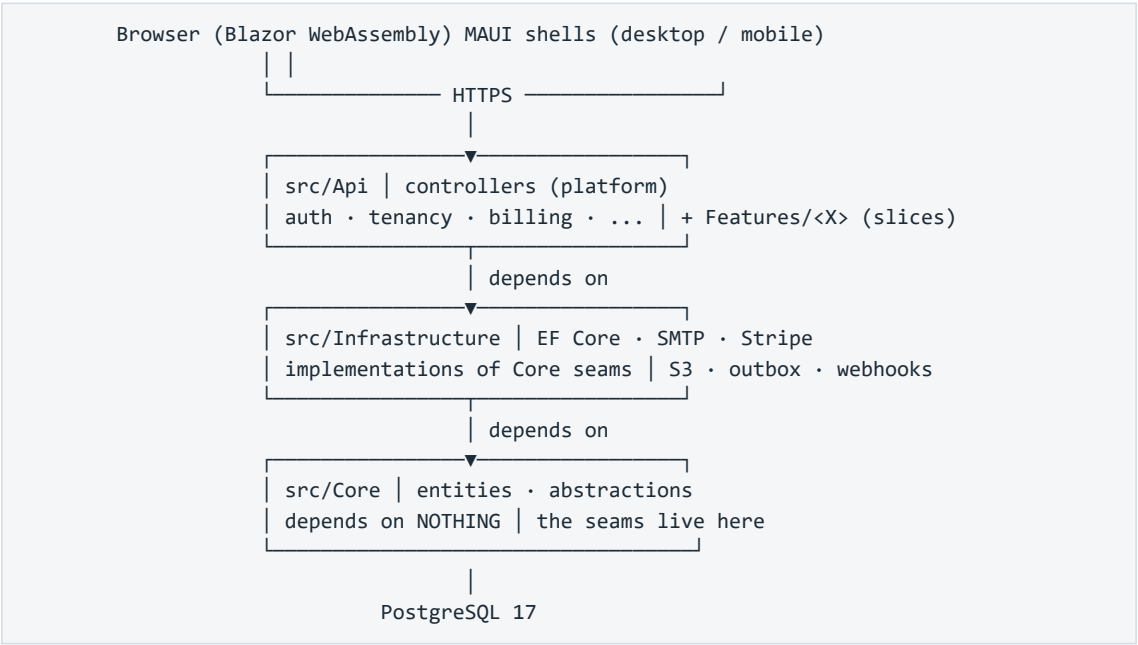
The companion repository. Alongside the book there is a companion repo built in teaching order, with one tagged checkpoint per lesson (`lesson-1.3`, `lesson-2.6`, …). Type everything yourself — that is the course — but when you are stuck, `git diff` your work against the checkpoint, or check it out and keep moving. Falling behind the typing is recoverable; skipping the understanding is not.

Don't skip Part 0. Lesson 0.2 pins your toolchain and infrastructure so that the other forty-nine lessons behave identically on your machine. Most abandoned courses die at "works on my machine"; we kill that failure mode first.

When a lesson cites a rule (R1–R35) or an ADR, that is a pointer into the reference platform's own governance documents — the machine-enforced invariants and the decision log. You are not just building an app; you are building the *system that keeps the app honest*, and those citations show you where each piece of it came from.

Reading order is dependency order. Later lessons lean on earlier seams without re-explaining them. If Part 5 feels steep, the fix is usually two parts back, not a re-read of Part 5.

The platform at a glance



One sentence to keep for the whole book: **app data belongs to the tenant, not the user** — and by the end of Part 2 you will have made it impossible, at three separate layers, for any line of feature code to violate that sentence.

Part 0 — Orientation

Lesson 0.1 — The mental model & the decision record

> **Part 0 · Orientation.** No code in this lesson — deliberately. You're about to type several hundred files over ~48 lessons. The two things that determine whether that makes you an architect or a very tired typist are (1) the mental model you hang each file on, and (2) the habit of recording *why* before *what*. Both are installed here.

Goal: you can draw the system from memory — its layers, its one non-negotiable invariant, and its unit of work — and you've written your project's first ADR.

Concepts & principles: multi-tenancy (tenant ≠ user); clean architecture's dependency rule; vertical slices vs. horizontal layers; Architecture Decision Records.

Maps to: ADR-C1/C2/C3, ADR-004 · `docs/DECISIONS.md`, `docs/WAYS_OF_WORKING.md` in the reference repo.

---

1. What you are going to build

A production-grade, multi-tenant SaaS **platform**: custom JWT auth (passwordless + OAuth + MFA), tenant isolation enforced by the type system, billing with Stripe, reliable async via a transactional outbox, RBAC, file storage, GDPR export/erasure, a public API, outbound webhooks, an admin back-office, observability — deployed to a real host at the end. Not a to-do app. The kind of chassis a real product bolts features onto.

You will build it **test-first, in vertical slices**, and every architectural rule will be enforced by a test you wrote — not by a comment hoping to be read.

2. The four ideas everything else hangs on

Idea 1 — Tenant ≠ user

The single most important sentence in this course: **app data belongs to the tenant, not the user.** A *tenant* is the paying unit — a household, a team, a company. Users are members of a tenant; they come and go, and the tenant's data stays. Get this backwards and you'll design tables, queries, and permissions around the wrong axis — and unwinding that later is a rewrite, not a refactor.

One consequence you'll meet in Lesson 2.6 and never stop meeting: every read and write of app data must be scoped to the current tenant, and that scoping must be **impossible to forget**, not merely encouraged.

Idea 2 — The dependency rule (clean architecture, minus the ceremony)

Three production layers, dependencies pointing strictly inward:

```
Api ──► Infrastructure ──► Core
(HTTP, auth, (EF Core, SMTP, (entities, abstractions —
 controllers, Stripe, S3, the depends on NOTHING)
 features) implementations)
```

`Core` holds the entities and the *seams* — small interfaces like `IEmailSender`, `IFileStorage`, `IBillingProvider`. `Infrastructure` implements them with real technology (MailKit, S3, Stripe). `Api` orchestrates. The payoff is swappability where it matters (Mailpit in dev, Brevo in prod — same seam) and testability everywhere. The UI (Blazor WebAssembly) is *just another API client* — it never touches the database.

Idea 3 — Clean platform + vertical slices (the hybrid)

Pure layered architecture scatters one feature across four folders. Pure feature-folders duplicate the platform in every slice. This codebase does both, deliberately, and the split is the point:

- The **platform** — auth, tenancy, billing, email, persistence — is the durable chassis,

organized in clean layers. Built once, reused by everything.

- Each **app feature** lives in ONE folder (`src/Api/Features/<Feature>/`) containing its

endpoints, handler, DTOs, and data contributor. Features *reuse* the platform; they never reach into each other.

When you wonder "where does this file go?", the question to ask is: *chassis or feature?*

Idea 4 — Vertical slices as the unit of work

Every increment of work cuts through all layers — DB to UI — and leaves the app working. You'll never build "all the tables" then "all the endpoints." Each lesson in this course is itself a slice: it ends with a green build, passing tests, and a tagged checkpoint.

3. The decision record — the habit that makes it stick

Here's what separates a codebase you can *maintain* from one you can only *fear*: six months from now, the question is never "what does this code do?" (you can read it), it's **"why is it like this — and is it safe to change?"**

An **Architecture Decision Record** answers that in ~10 lines, written at the moment of decision:

```
## ADR-002 — Custom JWT + rotating refresh tokens (not ASP.NET Core Identity)

**Date:** 2026-06-XX · **Status:** Accepted

**Context:** We need auth for web + native clients with passwordless and OAuth.
Identity brings cookie-centric defaults, schema we don't control, and hides the
token lifecycle we need to reason about (MFA step-up, rotation, native flows).

**Decision:** Issue our own short-lived JWT access tokens + rotating refresh
tokens; store only token hashes.

**Consequences:** We own token security (rotation, reuse detection — must test
it ourselves). In exchange: full control of claims (tenant_id drives data
isolation), one auth model across web/native, no framework fight when MFA lands.
```

Note what an ADR is *not*: not documentation of how the code works (the code shows that), and never deleted when wrong — a reversed decision gets a **new dated ADR** superseding the old one. The history of your reasoning is itself an asset.

The reference repo runs on this: 20 app ADRs + amendments in `docs/DECISIONS.md`. Every lesson ahead ends with an **Architecture Decision box** — the fork faced, the option chosen, the roads not taken. By Part 4 you should be predicting them before reading them; that reflex *is* the architectural skill this course exists to build.

4. Exercise — write your ADR-000 and ADR-001

Create your project directory (empty — the solution comes in Lesson 1.1) and start the decision log:

```
mkdir my-saas && cd my-saas && git init
mkdir docs && $EDITOR docs/DECISIONS.md
```

Write two ADRs yourself — don't copy:

1 **ADR-000 — Why I'm building this.** Context (what you want to learn), decision (build the platform from scratch, test-first), consequences (weeks of effort; deep ownership of every line).

1 **ADR-001 — Local secrets live in `.env`, never in the repo.** You haven't built the loading mechanism yet (that's 0.2 and 1.4) — which is exactly the point: **the decision precedes the code.** Context: secrets in committed files leak (git history is forever). Decision: a gitignored `.env` is the single local source of truth; a committed `.env.example` documents every key; production uses real environment variables. Consequences: one more setup step per machine; zero secrets in history — enforced by a scanner you'll add in the next lesson.

5. Checkpoint

```
git add docs/DECISIONS.md && git commit -m "docs: ADR-000 and ADR-001 — the decision log starts before the code"
git tag lesson-0.1
```

You should now be able to: state the tenant≠user invariant and its consequence for every future query; draw the three layers and their dependency direction; say which of the two organizational modes (chassis vs. feature folder) a given file belongs to; and write an ADR that captures context → decision → consequences in under a page.

Next: *Lesson 0.2 — A reproducible machine*, where your ADR-001 gets its mechanical teeth: `.env.example`, docker-compose, and a secret scanner.

Lesson 0.2 — A reproducible machine

> **Part 0 · Orientation.** Most from-scratch courses die at "install PostgreSQL." This lesson exists so yours can't. At the end, your machine runs the exact infrastructure the app will use for the next 47 lessons — pinned, containerized, verifiable with three commands — and your ADR-001 (secrets never in the repo) is enforced by machinery, not memory.

Goal: `docker compose up -d` gives you a healthy Postgres 17 and a mail trap; `dotnet --version` prints the pinned SDK; a secret can't reach a commit.

Concepts & principles: pinned toolchains ("latest stable, never previews" — and never "whatever was on the machine"); infrastructure-as-code for dev; the `.env` contract (ADR-001); fail-loud health checks; defense-in-depth for secrets.

Maps to: ADR-001, ADR-C13 · Foundation theme R20–R22 (the config contract this enables) · repo: `docker-compose.yml`, `.env.example`, `.gitignore`, `.gitleaks.toml`.

Prerequisites: 0.1 (your repo exists and ADR-001 is written).

---

1. Motivate — "works on my machine" is an architecture failure

Two developers clone the same repo. One has Postgres 15 installed as a Windows service from a project two jobs ago; the other has nothing. One gets a subtle JSON-function bug that doesn't reproduce; the other gets a connection refused. Neither failure is *in the code* — and that's the point: **your dev environment is part of your architecture**, and undeclared dependencies are its silent killers.

The fix is the same one you'll apply to code all course long: make the implicit explicit, and make violations loud.

2. Install the pinned toolchain

Three tools go on the machine itself; everything else runs in containers:

Tool	Version	Check
.NET SDK	**10.0.3xx** (the line this course pins)	`dotnet --version`
Docker Desktop / Engine	current stable	`docker version`
EF Core tooling	current stable	`dotnet tool install --global dotnet-ef` (first used in 1.3)

Pin the SDK **in the repo**, not in a README nobody reads — create `global.json`:

```
{
  "sdk": { "version": "10.0.301", "rollForward": "latestFeature" }
}
```

Now a machine with the wrong SDK fails at `dotnet build` with an explicit message instead of failing three lessons later with an inscrutable one. This is the course's recurring move — *fail*

loud and early — applied for the first time, to the toolchain itself.

3. Infrastructure as one file — docker-compose

The app needs PostgreSQL and — less obviously — an **SMTP trap**. From Lesson 2.3 onward the platform sends real email (magic links, OTPs, invitations), and you need somewhere for it to land in dev that isn't a real inbox. Mailpit accepts SMTP on one port and shows every message in a web UI on another — and, critically for Lesson 3.6, exposes an **HTTP API your tests will query** to fish out OTP codes. Two infrastructure decisions, one committed file — create `docker-compose.yml`:

```
services:
  db:
    image: postgres:17
    restart: unless-stopped
    environment:
      POSTGRES_DB: ${DB_NAME:-dev_db}
      POSTGRES_USER: ${DB_USER:-dev}
      POSTGRES_PASSWORD: ${DB_PASSWORD:-devpassword}
    ports:
      - "${DB_PORT:-5432}:5432"
    volumes:
      - db_data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U ${DB_USER:-dev} -d ${DB_NAME:-dev_db}"]
      interval: 5s
      timeout: 5s
      retries: 5
      start_period: 10s

  mail:
    image: axllent/mailpit:latest
    restart: unless-stopped
    ports:
      - "${MAIL_SMTP_PORT:-1025}:1025" # SMTP — the app sends here
      - "${MAIL_UI_PORT:-8025}:8025" # Web UI + API — you and your tests read here
    healthcheck:
      test: ["CMD-SHELL", "wget -q --spider http://localhost:8025 || exit 1"]
      interval: 5s
      timeout: 5s
      retries: 5
      start_period: 5s

volumes:
  db_data:
```

Read it as a set of decisions, because each line is one:

- **`postgres:17`** — a pinned major version, mirroring `global.json`'s job for the SDK.

"Latest stable, never previews" also means *never unpinned*.

- **`${DB_PORT:-5432}`** — every value is overridable via environment, with a working

default. A developer whose port 5432 is squatted by that old Windows service sets `DB_PORT=5433` in `.env` and moves on. (The reference repo defaults to 5433 for exactly that reason — port collisions are the #1 setup killer.)

- **`healthcheck`** — "container started" ≠ "Postgres accepting connections." Health

checks make readiness explicit, and anything that later `depends_on` these services waits for *healthy*, not merely *running*. You'll re-meet this exact idea as the API's own `/health` endpoint in Lesson 4.5.

- **`volumes: db_data`** — your data survives `docker compose down`. (Destroying it becomes a deliberate act: `docker compose down -v`.)

4. The `.env` contract — ADR-001 gets teeth

Your ADR-001 said: secrets live in a gitignored `.env`; a committed `.env.example` documents every key. Build both halves now.

`.gitignore` (start; it grows with the solution in 1.1):

```
.env
bin/
obj/
storage/
```

`.env.example` — **committed**, and it is *documentation with a job*:

```
# Copy this file to .env and fill in your values. .env is gitignored — never commit it.
# Single source of local-dev config + secrets: read by docker-compose (containers) AND
# by the API via DotNetEnv (from Lesson 1.4). See docs/DECISIONS.md ADR-001.

# ── Containers (docker-compose) ──────────────────────────
DB_NAME=dev_db
DB_USER=dev
DB_PASSWORD=devpassword
DB_PORT=5433

# Mailpit (SMTP trap)
MAIL_SMTP_PORT=1025
MAIL_UI_PORT=8025
```

Then `cp .env.example .env` and confirm `git status` does **not** list `.env`.

Notice the shape of the contract: *every* key the system reads appears here with an inline comment saying what it does, whether it's required, and what happens when unset. In Lesson 1.4 you'll write a test (`DocAndConfigSyncTests`) that **fails the build when a config key exists in code but not in this file** — turning "please document your config" from a review nag into a compile-adjacent guarantee (R20). The habit starts now.

Docker-compose reads `.env` automatically. So one file drives both the containers and — once DotNetEnv arrives in 1.4 — the app. Two consumers, one source of truth, zero drift.

The second layer: a secret scanner

`.gitignore` is advisory — it does nothing if someone hardcodes a connection string in a `.cs` file. Add gitleaks config, `.gitleaks.toml`:

```
# Secret scanning — the backstop behind ADR-001. Default rules + our allowlist.
[extend]
useDefault = true

[allowlist]
description = "Dev-only placeholder values that look like secrets but aren't"
paths = ['''\.env\.example''']
```

Run it locally (`gitleaks detect`) whenever you like; in Lesson 8.3 it becomes a CI gate. Layers, again: `.gitignore` prevents the accident, gitleaks catches what slips past, CI makes it policy. No single layer is trusted alone — the same defense-in-depth shape you'll apply to tenancy (filter + interceptor + arch test) from Part 2 on.

5. Run it — verify everything

```
docker compose up -d
docker compose ps # both services: status "healthy" (wait a few seconds)
dotnet --version # 10.0.3xx — matches global.json
docker compose exec db psql -U dev -d dev_db -c "select version();" # PostgreSQL 17.x
```

Open http://localhost:8025 — Mailpit's empty inbox. In Lesson 2.4, your app's first magic-link email lands here; in 3.6, your Playwright tests read OTP codes out of it through its API.

6. Architecture Decision

ARCHITECTURE DECISION

The fork: where does dev infrastructure live? (a) Installed natively per machine, (b) a shared cloud dev database, (c) containers defined in the repo.

Chosen: containers (ADR-C13). The infrastructure's *versions and topology* are code, reviewed like code; a new machine is productive in minutes; two projects with different Postgres majors coexist.

Rejected: native installs — unpinned, colliding, undocumented by nature. Shared cloud dev DB — couples every developer's iteration loop to a network and to each other's data (and costs money before the first feature exists).

The trade: Docker becomes a hard dev dependency. Accepted knowingly — it's also the test-isolation strategy (Testcontainers, Lesson 1.3) and the deployment unit (8.2), so the dependency pays three times.

7. Checkpoint

```
git add global.json docker-compose.yml .gitignore .gitleaks.toml .env.example
git commit -m "chore: reproducible dev environment — pinned SDK, compose infra, env
contract"
git tag lesson-0.2
```

You should now be able to: bring the full dev infrastructure up and down with one command; explain why the SDK and Postgres versions are pinned *in the repo* rather than in a README; state the `.env` / `.env.example` contract and the two layers that enforce it; and articulate why health checks matter before anything depends on these services.

Next: *Lesson 1.1 — Solution, projects & supply chain* — the .NET solution appears, with its dependencies pointing the right way and its packages locked.

Part 1 — The walking skeleton

Lesson 1.1 — Solution, projects & supply chain

> **Part 1 · The walking skeleton.** Nothing runs yet — that's next lesson. Here we lay out the solution so that the architecture from Lesson 0.1 isn't a diagram you promised to respect but a set of project references the compiler enforces. We also make a decision most projects defer until it hurts: exactly which versions of everything we depend on, pinned so a build today and a build in six months produce the same bytes.

Goal: a solution with every project in place, dependencies physically pointing inward, one central version table, and a locked, reproducible restore.

Concepts & principles: the dependency rule made mechanical; Central Package Management; lockfiles and supply-chain hygiene; warnings-as-errors from day one.

Maps to: ADR-C3/C4/C5 · R25–R27 · repo: `Perezosoft.slnx`, `Directory.Build.props`, `Directory.Packages.props`, `*/*.csproj`.

Prerequisites: 0.2 (pinned SDK; `dotnet --version` prints 10.0.3xx).

---

1. Motivate — the layout is the first enforcement

Every codebase claims to have an architecture. The honest question is: *what happens if someone violates it?* If the answer is "a reviewer might notice," you have a convention. If the answer is "it does not compile," you have a structure.

Project references are the cheapest structural enforcement .NET offers. When `Core` has no reference to `Infrastructure`, a domain entity *cannot* reach out to EF Core or SMTP — not "should not," cannot. The dependency rule from Lesson 0.1 stops being a poster on the wall the moment the reference graph encodes it. That's what we build in this lesson.

The second problem we solve is quieter and nastier. `dotnet add package` with no version pinned means your build depends on *when* you ran it. Two developers restore on different days and get different transitive graphs; CI builds an artifact subtly unlike the one you tested; and when a compromised package version hits NuGet — it has happened — nothing in your build even notices. Reproducibility isn't a nicety; it's the precondition for being able to say "this is what we shipped."

2. Create the skeleton

The solution uses the modern XML solution format (`.slnx` — human-readable, merge-friendly, supported by .NET 10 tooling):

```
dotnet new sln --format slnx -n Perezosoft
dotnet new classlib -o src/Core -n Perezosoft.Core
dotnet new classlib -o src/Infrastructure -n Perezosoft.Infrastructure
dotnet new web -o src/Api -n Perezosoft.Api
dotnet new blazorwasm -o src/Web -n Perezosoft.Web
dotnet new razorclasslib -o src/Shared.Ui -n Perezosoft.Shared.Ui
dotnet new xunit -o tests/Core.Tests -n Perezosoft.Core.Tests
dotnet new xunit -o tests/Api.Tests -n Perezosoft.Api.Tests
dotnet new nunit -o tests/E2E.Tests -n Perezosoft.E2E.Tests
dotnet sln add src/Core src/Infrastructure src/Api src/Web src/Shared.Ui \
              tests/Core.Tests tests/Api.Tests tests/E2E.Tests
```

Delete the template's sample classes (`Class1.cs`, the Blazor counter page can stay for now — it goes when the real UI arrives in 3.4). Then wire the references — this is the architecture, so read it slowly:

```
dotnet add src/Infrastructure reference src/Core # Infra implements Core seams
dotnet add src/Api reference src/Core src/Infrastructure # Api composes both
dotnet add src/Web reference src/Shared.Ui # Web hosts the shared UI
dotnet add tests/Core.Tests reference src/Core
dotnet add tests/Api.Tests reference src/Api src/Infrastructure src/Core
```

What's deliberately **absent** is the point: `Core` references *nothing* of ours. `Infrastructure` cannot see `Api`. `Web` cannot see `Core`, `Infrastructure`, or `Api` at all — the UI will talk to the API over HTTP like any other client (Golden Rule: the clean API boundary), and the compiler now holds it to that. Try it: add `using Perezosoft.Infrastructure;` to a file in `Core` — it does not build. That error message is the architecture working.

Each project file stays nearly empty. Here is the real `Perezosoft.Core.csproj` in its entirety:

```
<Project Sdk="Microsoft.NET.Sdk">

  <PropertyGroup>
    <TargetFramework>net10.0</TargetFramework>
    <ImplicitUsings>enable</ImplicitUsings>
    <Nullable>enable</Nullable>
  </PropertyGroup>

</Project>
```

Nine lines. Everything else lives at the solution root, where it applies to *all* projects at once — which is our next move.

3. One build policy for the whole solution

MSBuild reads a file named `Directory.Build.props` from every ancestor directory of a project. Put one at the repo root and every project inherits it — a single place for build policy, impossible for a new project to forget. This is the reference platform's actual file:

```
<Project>

  <!--
    Solution-wide build hygiene. Nullable reference types on, and warnings are
    errors so the invariants the analyzers and the compiler check can't rot into
    ignored noise. This is half of "solid by construction" — the architecture
    tests are the other half, and CI runs both.
  -->
  <PropertyGroup>
    <Nullable>enable</Nullable>
    <TreatWarningsAsErrors>true</TreatWarningsAsErrors>
    <ImplicitUsings>enable</ImplicitUsings>
    <!--
      Supply-chain hardening. Restore writes a packages.lock.json next to each
      project pinning the full transitive graph; those files are committed and CI
      restores in locked mode so any drift fails the build. Paired with Central
      Package Management (Directory.Packages.props).
    -->
    <RestorePackagesWithLockFile>true</RestorePackagesWithLockFile>
  </PropertyGroup>

</Project>
```

Two of these settings deserve a hard look, because both are *decisions about failure*:

`TreatWarningsAsErrors` — a warning is the compiler telling you something is probably wrong. On a codebase that tolerates warnings, the count creeps from 3 to 300, and the one that mattered drowns. Turning warnings into errors on day one costs nothing (there is no code yet!) and means the count stays at zero forever. Adopting this on an *existing* codebase is a month of cleanup — which is exactly why it must happen now, in the empty solution. Several rules you'll meet later (like nullability soundness) are only real because a violation refuses to compile.

`Nullable` — nullable reference types turn "I forgot this could be null" from a production `NullReferenceException` into a compile-time squiggle — and with the previous setting, into a build failure. The domain modeling you'll do from Part 2 onward leans on this constantly: `string?` in an entity is a *documented decision* that the value is optional, checked by the compiler, rather than tribal knowledge.

4. Central Package Management — one version table

Without CPM, every `.csproj` carries its own `<PackageReference Include="X" Version="Y">` and the same package drifts across projects (the reference repo actually hit this: two test projects referencing different versions of the test SDK). With CPM, versions live in **one** committed table, `Directory.Packages.props`, and project files reference packages *without* versions. An excerpt of the real table:

```
<Project>
  <PropertyGroup>
    <ManagePackageVersionsCentrally>true</ManagePackageVersionsCentrally>
  </PropertyGroup>

  <ItemGroup>
    <!-- App / API -->
    <PackageVersion Include="DotNetEnv" Version="3.2.0" />
    <PackageVersion Include="Microsoft.AspNetCore.Authentication.JwtBearer"
Version="10.0.9" />

    <!-- Infrastructure -->
    <PackageVersion Include="MailKit" Version="4.17.0" />
    <PackageVersion Include="Npgsql.EntityFrameworkCore.PostgreSQL" Version="10.0.2" />

    <!-- Tests -->
    <PackageVersion Include="xunit" Version="2.9.3" />
    <PackageVersion Include="Testcontainers.PostgreSql" Version="4.12.0" />
    <!-- Consolidated: E2E referenced 17.14.0, the xUnit projects 17.14.1 → highest wins.
-->
    <PackageVersion Include="Microsoft.NET.Test.Sdk" Version="17.14.1" />
  </ItemGroup>
</Project>
```

Create yours with just the packages the walking skeleton needs (the test SDK, xunit); every later lesson that says "add package X" means: add the `<PackageVersion>` row here, add a version-less `<PackageReference>` in the project that uses it. One table to review when you upgrade; one diff line when a version changes; zero possibility of the same package at two versions (that's rule R27, and it's enforced by construction).

Notice that comment about `17.14.0` vs `17.14.1` — it's a fossil of the exact drift CPM exists to prevent, found *during* the consolidation and preserved in the reference repo as a comment. Real codebases carry these scars; good ones label them.

5. Lock it — restore becomes reproducible

`RestorePackagesWithLockFile` (already in our props file) makes restore write a `packages.lock.json` beside each project: the **full transitive closure** — every package your packages depend on, recursively, with exact versions and content hashes. Commit these files. Then the magic flag:

```
dotnet restore --use-lock-file # first restore: writes the lockfiles
git add **/packages.lock.json
dotnet restore --locked-mode # from now on (and in CI): verify, don't resolve
```

In locked mode, restore doesn't *resolve* versions — it *verifies* them. If anything in the graph differs from the lockfile (a floating version moved, a transitive dependency was swapped, a hash doesn't match), restore **fails loudly** instead of silently building something new. When lesson 1.6 stands up CI, `--locked-mode` is one of its first gates: a supply-chain change becomes a red build and a reviewable diff, not an invisible drift.

The mental model: your dependency graph is now *code*. It changes by commit, with review, or not at all.

6. Run it

```
dotnet build
# 0 Warning(s), 0 Error(s) — and it must stay that way: warnings ARE errors now
dotnet restore --locked-mode
# succeeds — graph matches the committed lockfiles
```

And prove a gate actually gates (pain now, so you trust it later): edit a `packages.lock.json` by hand — change any version digit — and run `dotnet restore --locked-mode` again. It fails with `NU1004`. Revert. That failure is what a supply-chain surprise will look like: loud, early, and in your terminal instead of in production.

7. Architecture Decision

ARCHITECTURE DECISION

The fork: how do we manage dependency versions? (a) Per-project references with whatever versions were current at add-time; (b) a shared props file with version *variables*; (c) Central Package Management + committed lockfiles + locked-mode CI.

Chosen: (c) — CPM for one reviewable version table (R25), lockfiles for the transitive graph, locked restore so drift fails the build (R27). Also decided here: warnings-as-errors and nullable from the empty solution, because both are nearly free now and prohibitively expensive to retrofit.

Rejected: (a) is how version drift and silent supply-chain changes happen — it's the default, which is the problem. (b) centralizes the *numbers* but not the *graph*: transitive dependencies still float.

The trade: every dependency bump is now an explicit two-file diff (table + lock), and a new package is a two-step add. Accepted: that friction *is* the review point.

8. Checkpoint

```
git add -A && git commit -m "feat: solution skeleton — projects, inward refs, CPM, locked restore"
git tag lesson-1.1
```

You should now be able to: explain why `Core` referencing nothing is the architecture's first enforcement; add a package the CPM way; say what a lockfile pins that a version number doesn't; and articulate why warnings-as-errors is a day-one decision, not a someday cleanup.

Next: *Lesson 1.2 — First endpoint, first test* — the API says hello, and the Red→Green loop you'll live fifty times gets its first turn.

Lesson 1.2 — First endpoint, first test

> **Part 1 · The walking skeleton.** The solution compiles but does nothing. By the end of this lesson it answers HTTP — and, more importantly, a test *proves* it answers HTTP by booting the real application. This is the lesson where you live the Red→Green→Refactor loop for the first time, on the smallest possible stakes, so that when the stakes are tenant isolation or billing you already trust the rhythm.

Goal: `GET /health` returns 200, and a test in `Api.Tests` boots the actual app in-process and asserts it.

Concepts & principles: the walking skeleton; Red→Green→Refactor; in-process integration testing with `WebApplicationFactory`; testing the real composition, not a hand-assembled copy.

Maps to: ADR-C14 · repo: `src/Api/Program.cs`, `tests/Api.Tests/Infrastructure/IntegrationTestFactory.cs`, `tests/Api.Tests/Integration/HarnessSmokeTests.cs`, `src/Api/Perezosoft.Api.http`.

Prerequisites: 1.1 (solution builds, CPM in place).

---

1. Motivate — why a "hello world" deserves a whole lesson

A *walking skeleton* is the thinnest possible slice that exercises the whole path: request in, response out, test proving it. It looks trivial. It isn't, because it forces every piece of plumbing to exist and cooperate **before** any feature depends on that plumbing: the web host boots, routing routes, the test project can reach the app, CI (next lessons) can run it all. Every integration problem you flush out now is one that won't ambush you inside a complicated lesson later.

There's also a decision hiding in "a test proving it" — *what kind* of test? We could unit-test a handler function and call it a day. But the failures that hurt in web apps are rarely inside one function; they're in the *composition* — DI registrations missing, middleware ordered wrong, routes not mapped, config not read. So our default harness boots the **real** `Program` — the actual composition root, not a hand-assembled lookalike — in-process, and talks to it over real HTTP semantics. The reference platform's harness makes the reasoning explicit; from its actual doc comment:

> Boots the REAL app (`Program`) via `WebApplicationFactory<T>` … so tests exercise the full pipeline …, not hand-assembled services. … [this] de-risks routing/DI refactors by asserting routes stay stable end-to-end.

A test against a hand-wired `ServiceCollection` keeps passing while the real app is broken. A test against `Program` cannot.

2. Red — the failing test comes first

In `tests/Api.Tests`, add the package references (CPM style — versions come from the table you made in 1.1): `Microsoft.AspNetCore.Mvc.Testing`, and reference the Api project. Then write the test **before** the endpoint exists:

```
// tests/Api.Tests/HealthEndpointTests.cs
using System.Net;
using Microsoft.AspNetCore.Mvc.Testing;

namespace Perezosoft.Api.Tests;

public class HealthEndpointTests : IClassFixture<WebApplicationFactory<Program>>
{
    private readonly WebApplicationFactory<Program> _factory;

    public HealthEndpointTests(WebApplicationFactory<Program> factory) => _factory =
factory;

    [Fact]
    public async Task Health_endpoint_responds_200()
    {
        var client = _factory.CreateClient(); // in-process; no port, no network

        var response = await client.GetAsync("/health");

        Assert.Equal(HttpStatusCode.OK, response.StatusCode);
    }
}
```

Run it:

```
dotnet test tests/Api.Tests
# Failed! Health_endpoint_responds_200
# Assert.Equal() Failure: Expected OK, Actual NotFound
```

Red — the app boots (WebApplicationFactory found `Program` and started the real host) but there is no `/health` endpoint, so the request falls through to a 404. That's a *properly* failing test: it fails for exactly the reason it should, and the failure message describes the missing behavior.

One wrinkle to know about before it bites: our `Program.cs` is top-level statements (no explicit class), and the class the compiler generates for it is **internal**. On current SDKs the web template arranges for the test host to reach it anyway, so the test above compiles and runs — but if you ever hit `error CS0246: The type or namespace name 'Program' could not be found`, the fix is one line in `Perezosoft.Api.csproj`, and the reference platform carries it as standing hygiene:

```
<ItemGroup>
  <InternalsVisibleTo Include="Perezosoft.Api.Tests" />
</ItemGroup>
```

`InternalsVisibleTo` is a scalpel: it grants *one named assembly* access to internals, instead of making `Program` public for the whole world. You'll see this same "smallest-possible opening" instinct again and again — it's the security posture of the entire platform in miniature. Add it now even though the build didn't force you to; the next person's SDK might.

3. Green — the minimum that passes

`src/Api/Program.cs`, in full at this stage of its life:

```
var builder = WebApplication.CreateBuilder(args);

builder.Services.AddHealthChecks();

var app = builder.Build();

app.MapHealthChecks("/health");

app.Run();
```

```
dotnet test tests/Api.Tests
# Passed! - 1 test
```

That's the loop: a failing test that *specifies*, then the minimum code that *satisfies*. Resist the urge to add more — no controllers, no Swagger, no logging config. Every one of those will arrive in the lesson where a test demands it. (For the curious: this file grows to ~300 lines by Part 8 in the reference repo, and every block of it traces to a lesson.)

Why `AddHealthChecks()`/`MapHealthChecks` rather than a hand-rolled `app.MapGet("/health", …)`? Because this endpoint has a *career* ahead of it: in Lesson 4.5 it splits into liveness (`/health`) and readiness (`/health/ready`) with a real database probe behind a tag filter, and in Lesson 8.3 the deploy pipeline's smoke gate calls it to decide whether a release goes out. Using the framework's health-check pipeline from day one means that growth is additive. The reference repo's final form, for a preview of where this line of code is headed:

```
app.MapHealthChecks("/health", new HealthCheckOptions { Predicate = _ => false });
app.MapHealthChecks("/health/ready", new HealthCheckOptions { Predicate = check =>
check.Tags.Contains("ready") });
```

4. Refactor & harden — a scratchpad and a habit

No production refactor needed yet (there's barely any production). But two small work-habits start here.

The `.http` scratchpad. Create `src/Api/Perezosoft.Api.http` — using the https port from *your* `src/Api/Properties/launchSettings.json` (`dotnet new web` assigns a random one; the reference repo happens to sit on 7160):

```
@host = https://localhost:7160

GET {{host}}/health
```

Visual Studio and VS Code (REST Client) execute these files in-editor. Every lesson that adds an endpoint adds a request here — by Part 7 it's a living, executable catalog of the whole API surface, versioned with the code. It's the cheapest API documentation that can't go stale, because you *use* it to poke the app.

Run the real thing. Tests passing in-process is necessary, not sufficient — actually boot it once:

```
dotnet run --project src/Api
# listening on https://localhost:xxxx
curl -k https://localhost:xxxx/health
# Healthy
```

The response body `Healthy` comes from the health-check middleware's default writer. In-process tests + a real boot = the two views you'll keep cross-checking all course.

5. Run it

```
dotnet build # 0 warnings, 0 errors — still, and always
dotnet test tests/Api.Tests
# 1 passed
```

6. Architecture Decision

ARCHITECTURE DECISION

The fork: what does the default test harness boot? (a) Nothing — unit-test handlers as plain functions; (b) a hand-assembled `ServiceCollection` with the pieces each test needs; (c) the real `Program` via `WebApplicationFactory`, in-process.

Chosen: (c) as the default for anything HTTP-shaped (ADR-C14's integration layer). The composition root is where web apps actually break — missing registrations, middleware order, unmapped routes — and only booting the real one tests it. In-process means no ports, no network flakes, parallel-safe.

Rejected: (a) alone leaves the wiring untested (we still unit-test pure logic — Core.Tests exists for exactly that). (b) is the seductive trap: it *looks* like an integration test but silently diverges from the real app — a test passing against a copy proves nothing about the original.

The trade: booting the real app makes some things awkward on purpose — config the app reads at startup must genuinely exist (you'll feel this in 1.4, and the reference harness has a documented seam for it: the JWT secret must be an *environment variable* because `Program` reads it before the factory's config hooks run). That awkwardness is information: it's your startup contract made visible.

7. Checkpoint

```
git add -A && git commit -m "feat: walking skeleton — health endpoint + real-app test
harness"
git tag lesson-1.2
```

You should now be able to: explain what a walking skeleton de-risks; write the failing test first and feel why the order matters; say why `WebApplicationFactory<Program>` beats a hand-assembled service collection; and explain what `InternalsVisibleTo` grants and why it's preferable to making things public.

Next: *Lesson 1.3 — Database & the test container* — Postgres joins the skeleton, and the tests get a real database that appears and disappears on demand.

Lesson 1.3 — Database & the test container

> **Part 1 · The walking skeleton.** The skeleton gets a spine: PostgreSQL via EF Core, a first migration, and — the real prize — a test fixture that gives every test run a genuine Postgres that appears in seconds and vanishes without trace. The single most consequential testing decision of the course happens here: **we do not use the EF in-memory provider. Ever.** By the end you'll know exactly why.

Goal: `AppDbContext` exists with one throwaway entity, a migration creates its schema, and a test writes to and reads from a *real* Postgres spun up by the test run itself.

Concepts & principles: EF Core migrations discipline; Testcontainers; why in-memory database fakes lie; model-derived test resets (isolation that can't be forgotten).

Maps to: ADR-C6/C7 · R11 · repo: `src/Infrastructure/Persistence/AppDbContext.cs`, `src/Api/AppDbContextFactory.cs`, `tests/Api.Tests/Infrastructure/PostgresFixture.cs`, `tests/Api.Tests/Infrastructure/PostgresTestBase.cs`, `tests/Api.Tests/MigrationsTests.cs`.

Prerequisites: 1.2 (harness runs); 0.2 (Docker running — Testcontainers needs the daemon, and you already have it for compose).

---

1. Motivate — your tests are only as honest as their database

Here's the trap nearly every .NET team falls into once. EF Core ships an in-memory provider; it's right there; tests using it need no Docker and run fast. So the test suite grows on it. Then one day a query that passed a thousand test runs fails in production, because the in-memory provider isn't a database — it's a `List<T>` wearing a trench coat. The reference platform's fixture states the consequence as a matter of record, in its actual doc comment:

> Real Postgres (not the EF in-memory provider) is required because several services branch on `Database.IsRelational()` and use relational-only constructs — `ExecuteUpdateAsync`, savepoints — that the in-memory provider silently skips.

"Silently skips" is the killer phrase. And the list is longer than that comment: the in-memory provider has no transactions worth the name, no unique constraints, no `SKIP LOCKED` (Lesson 4.1 depends on it), case-insensitive string comparisons where Postgres is case-sensitive — and, fatally for this course, its handling of **global query filters** (Lesson 2.6, the keystone of tenant isolation) diverges from a real relational provider. A tenant-isolation test that passes in-memory proves *nothing*.

The classical escape was a shared "test database" on somebody's server — slow to reset, polluted by yesterday's runs, fought over by CI agents. Testcontainers dissolves the dilemma: the test run itself starts a pristine `postgres:17` in a Docker container, uses it, and destroys it. Real engine, zero residue.

2. The context and its first entity

In `Infrastructure`, add the EF packages (CPM rows + version-less references — `Npgsql.EntityFrameworkCore.PostgreSQL`; `Microsoft.EntityFrameworkCore.Design` goes in the Api project for tooling). Then the context — humble at this stage:

```
// src/Infrastructure/Persistence/AppDbContext.cs
using Microsoft.EntityFrameworkCore;

namespace Perezosoft.Infrastructure.Persistence;

public class AppDbContext : DbContext
{
    public AppDbContext(DbContextOptions<AppDbContext> options) : base(options) { }

    protected override void OnModelCreating(ModelBuilder builder)
    {
        base.OnModelCreating(builder);
        // Per-entity mapping lives in one IEntityTypeConfiguration<TEntity> class each,
        // discovered from this assembly — no central registry to forget.
        builder.ApplyConfigurationsFromAssembly(typeof(AppDbContext).Assembly);
    }
}
```

That `ApplyConfigurationsFromAssembly` line is a pattern you met in 0.2 (health checks) and will meet a dozen more times: **discovery over registration**. Entity configurations will be *found* because they exist, not because someone remembered to list them. (In 2.6, tenant filters attach the same way — by reflection over the model — and in this lesson's fixture, the reset list will too. One idea, three appearances already.)

For a first entity we need something disposable — the real platform's first entities (User, Tenant) arrive with Part 2 and deserve their own lessons. Use a placeholder `Probe` entity with an `Id` and a `Note` — its only job is to prove the pipeline, and you'll delete it in 2.1 without ceremony. Give it an `IEntityTypeConfiguration<Probe>` so the discovery pattern gets exercised.

Now scroll your build output before moving on — there's a landmine here worth defusing in daylight. Depending on package versions, adding the EF stack can produce:

```
warning MSB3277: Found conflicts between different versions of
"Microsoft.EntityFrameworkCore" that could not be resolved.
```

…and the build still says **succeeded**. Didn't we make warnings errors in 1.1? We did — for *compiler* warnings. `MSB3277` is an **MSBuild** warning, a different pipeline that `TreatWarningsAsErrors` does not cover. Your zero-warnings bar has a blind spot, and this is the classic thing that hides in it: the Npgsql provider pulling one EF Core version while the Design/test packages pull another. The fix is to pin the shared assembly explicitly — add `Microsoft.EntityFrameworkCore.Relational` at the same version as your other EF packages to the CPM table and reference it from `Api.Tests` (the reference repo carries exactly this pin, with a comment saying why). The transferable lesson: **"the build is green" and "the build is clean" are different claims** — read the output when you add a dependency, because version-unification warnings are how runtime `MethodNotFound` surprises introduce themselves politely first.

3. Migrations — schema changes as reviewable code

The EF tooling is a separate global tool, not part of the SDK — install it once (0.2's toolchain table now has a third row):

```
dotnet tool install --global dotnet-ef
dotnet ef migrations add InitialCreate --project src/Infrastructure --startup-project
src/Api
```

...and it fails, or worse, half-works, because EF's design-time tooling wants to *boot your app* to obtain a configured `DbContext` — and your app (from 1.4 onward) will refuse to boot without validated config. The reference platform closes this with a **design-time factory**, and its doc comment explains the whole story:

> Design-time factory so EF tooling ... can build the context WITHOUT booting the API host — routing through the host runs startup config validation (e.g. `Jwt:Secret`) that isn't present at design time. The connection string comes from `ConnectionStrings__DefaultConnection`, the SAME single source the running app uses ... There is no hardcoded fallback.

```
// src/Api/AppDbContextFactory.cs (abridged from the real file)
public class AppDbContextFactory : IDesignTimeDbContextFactory<AppDbContext>
{
    public AppDbContext CreateDbContext(string[] args)
    {
        try { DotNetEnv.Env.TraversePath().Load(); } catch { /* no .env present */ }

        var connectionString =
            Environment.GetEnvironmentVariable("ConnectionStrings__DefaultConnection")
            ?? throw new InvalidOperationException(
                "ConnectionStrings__DefaultConnection is not set. Add it to .env ...");

        var options = new DbContextOptionsBuilder<AppDbContext>()
            .UseNpgsql(connectionString).Options;
        return new AppDbContext(options);
    }
}
```

Note what it *refuses* to do: no hardcoded `Host=localhost;...` fallback. The connection string lives in exactly one place — your `.env` (ADR-001, given teeth in 0.2) — and if it's missing, the error says precisely what to fix. A fallback would "work" and thereby hide misconfiguration until it mattered.

Which means two files gain a line right now, honoring 0.2's contract that every key the system reads is documented. In `.env` (so the tooling works on your machine) and in `.env.example` (so it works on the next machine):

```
# REQUIRED — Postgres connection for the API + EF tooling. Mirror the DB_* values above
# (note the port: 5433 if you kept 0.2's collision-avoiding default).
ConnectionStrings__DefaultConnection="Host=localhost;Port=5433;Database=dev_db;Username=dev
;Password=devpassword"
```

(When 1.4's doc-sync gate arrives, an undocumented key like this would fail the build — we're just early.)

Run the migration command again; a `Migrations/` folder appears in Infrastructure. Read the generated file once, top to bottom — you'll rarely need to again, but you should know what the tool writes on your behalf: an `Up()` building your tables, a `Down()` reversing it, and a model snapshot the *next* migration diffs against. Migrations are why "the schema" is never a rumor: it's the sum of committed, reviewed, replayable scripts.

4. The fixture — a real database that can't leak between tests

Now the centerpiece. Add `Testcontainers.PostgreSql` to `Api.Tests` and build the fixture (this is the reference platform's real code, trimmed of the tenancy parts that arrive in Part 2):

```
// tests/Api.Tests/Infrastructure/PostgresFixture.cs
public sealed class PostgresFixture : IAsyncLifetime
{
    private readonly PostgreSqlContainer _container = new
PostgreSqlBuilder("postgres:17").Build();

    public string ConnectionString => _container.GetConnectionString();

    public async Task InitializeAsync()
    {
        await _container.StartAsync();
        await using var db = CreateContext();
        // Build the schema from the EF model (fast; no migration history needed for
tests).
        await db.Database.EnsureCreatedAsync();
    }

    public Task DisposeAsync() => _container.DisposeAsync().AsTask();

    public AppDbContext CreateContext()
    {
        var options = new DbContextOptionsBuilder<AppDbContext>()
            .UseNpgsql(ConnectionString).Options;
        return new AppDbContext(options);
    }

    /// <summary>Truncates every table so each test starts from a clean slate. The table
    /// list is DERIVED from the EF model, so a new entity is reset automatically — no
    /// hand-maintained list to forget.</summary>
    public async Task ResetAsync()
    {
        await using var db = CreateContext();
        var tables = db.Model.GetEntityTypes()
            .Select(e => e.GetTableName())
            .Where(name => name is not null)
            .Distinct()
            .Select(name => $"\"{name}\"");
        var sql = "TRUNCATE TABLE " + string.Join(", ", tables) + " RESTART IDENTITY
CASCADE;";
        await db.Database.ExecuteSqlRawAsync(sql);
    }
}

[CollectionDefinition(PostgresCollection.Name)]
public sealed class PostgresCollection : ICollectionFixture<PostgresFixture>
{
    public const string Name = "postgres";
}
```

Three design decisions in forty lines, each load-bearing:

One container per run, not per test. Container startup costs seconds; a suite of hundreds of tests can't pay it each time. The xUnit *collection fixture* shares one container across every test class marked `[Collection(PostgresCollection.Name)]`, and xUnit runs a collection's members serially — so sharing is safe by construction.

Reset between tests, and make the reset un-forgettable. Shared container means shared state, so every test must start clean. The rookie version is a hand-maintained list of tables to truncate — which goes stale the day someone adds an entity and forgets the list (this exact bug is why rule R11 exists; the reference repo found it in an audit). The fix is the discovery pattern again: **derive the table list from `db.Model.GetEntityTypes()`**. New entity → automatically in the reset. And rather than trusting each test to call `ResetAsync()`, a tiny base class makes isolation structural:

```
// tests/Api.Tests/Infrastructure/PostgresTestBase.cs — real file, in full
public abstract class PostgresTestBase(PostgresFixture fixture) : IAsyncLifetime
{
    protected PostgresFixture Fixture { get; } = fixture;

    public Task InitializeAsync() => Fixture.ResetAsync(); // before EVERY test
    public Task DisposeAsync() => Task.CompletedTask;
}
```

Inherit it and there is no reset call to forget. Same move as the tenant filter to come: correct by default, not correct by discipline.

`EnsureCreatedAsync` here, migrations elsewhere. The fixture builds schema straight from the model — fast, no history table. But that means the *migrations themselves* could rot untested. So one more test exists solely to apply every migration, in order, against a scratch database (`MigrationsTests` in the reference repo — and the integration harness from 1.2 will boot the app with its real startup-migration path in Part 2). Schema-from-model for speed; migrations proven separately; nothing unverified.

5. Red → Green — prove the loop against real Postgres

```
// tests/Api.Tests/ProbePersistenceTests.cs
[Collection(PostgresCollection.Name)]
public class ProbePersistenceTests(PostgresFixture fixture) : PostgresTestBase(fixture)
{
    [Fact]
    public async Task Probe_roundtrips_through_real_postgres()
    {
        await using (var db = Fixture.CreateContext())
        {
            db.Add(new Probe { Note = "hello from a container" });
            await db.SaveChangesAsync();
        }
        await using (var db2 = Fixture.CreateContext()) // fresh context: no cache tricks
        {
            var probe = await db2.Set<Probe>().SingleAsync();
            Assert.Equal("hello from a container", probe.Note);
        }
    }
}
```

```
dotnet test tests/Api.Tests
# Passed! - 2 tests (health + probe) ... first run pulls postgres:17, be patient
```

Watch `docker ps` during the run: a postgres container blinks into existence, then is gone. No cleanup script, nothing to remember, nothing left behind.

6. Architecture Decision

ARCHITECTURE DECISION

The fork: what database do tests run against? (a) EF in-memory provider; (b) SQLite in-memory ("at least it's relational"); (c) a shared long-lived test server; (d) Testcontainers — throwaway real Postgres per run.

Chosen: (d), with schema-from-model for speed and a separate migrations-apply test. Tests must exercise the engine production runs, because this platform *leans* on Postgres-specific behavior: global query filters as the tenancy wall, `SKIP LOCKED` job claiming, savepoints, and eventually row-level security. (ADR-C7/C13; R11 for the derived reset.)

Rejected: (a) silently skips the exact constructs we depend on — a green suite proving nothing. (b) is a different engine with different SQL, different concurrency, no RLS: it converts "provider lies" into "dialect lies". (c) reintroduces shared mutable state across runs and machines — the thing test isolation exists to kill.

The trade: Docker becomes a hard requirement for running the suite, and the first test run pays an image pull. Accepted consciously in 0.2 — the same daemon already runs your dev infrastructure, and CI (1.6) has it natively.

7. Checkpoint

```
git add -A && git commit -m "feat: EF Core + Postgres — first migration, Testcontainers fixture, model-derived reset"
git tag lesson-1.3
```

You should now be able to: list three concrete ways the in-memory provider lies; explain the container-per-run / reset-per-test economy and why the reset list is derived, not written; say what a design-time factory is for and why it must not have a connection fallback; and describe how migrations stay verified even though the fixture doesn't use them.

Next: *Lesson 1.4 — Configuration & the options pattern* — the app learns to read config, validate it at startup, and fail loudly at boot rather than mysteriously at 3 a.m.

Lesson 1.4 — Configuration & the options pattern

> **Part 1 · The walking skeleton.** The app can answer HTTP and talk to Postgres; now it learns to be *configured* — and, more to the point, to refuse to start when it isn't. Configuration is where "works on my machine" goes to hide: a missing key that defaults to something plausible, a secret pasted into a JSON file, a setting the code reads that nobody documented. This lesson closes all three holes, and closes them with machinery, not vigilance.

Goal: the API loads `.env` in dev, exposes typed, validated settings objects, fails at *startup* (not first-use) on missing/invalid config, and a test fails the build when code reads a config key that isn't documented.

Concepts & principles: the config hierarchy; secrets never in the repo (ADR-001 mechanized); typed settings with construction-time validation; fail-fast startup; the doc-sync gate.

Maps to: ADR-001 · R20–R22 · repo: `src/Api/Configuration/SettingsProvider.cs`, `SettingsRegistration.cs`, `src/Api/Program.cs` (the DotNetEnv line), `tests/Api.Tests/DocAndConfigSyncTests.cs`, `.env.example`, `appsettings.json`.

Prerequisites: 1.3; the `.env` contract from 0.2.

---

1. Motivate — the 3 a.m. config bug

Picture the failure mode we're designing against. A service reads `config["Jwt:Secret"]` deep inside a request handler. On the machine where it's missing, nothing happens at startup — the app boots, health checks pass, the deploy is declared good. Hours later the first user tries to sign in and gets a 500, and the stack trace points at token-signing code, three layers away from the actual problem: a key nobody set. Now multiply by every config value in the system.

The root defect isn't the missing key — machines will always occasionally be misconfigured. The defect is **when** and **how** the system tells you. Config errors should surface at *boot*, in a message that names the key and says where to set it. That transformation — from runtime mystery to startup message — is what this lesson builds.

And beneath it, a second rule from Lesson 0.1's ADR-001: secrets never live in committed files. Which raises the practical question: where *does* the app get them, and how do committed files and secret files cooperate?

2. The config hierarchy — one mental model

ASP.NET Core builds `IConfiguration` from layered sources, later layers overriding earlier ones. Our arrangement (dev):

```
appsettings.json committed — non-secret defaults, shape documentation
appsettings.Development.json committed — dev-only tweaks (verbose logging, etc.)
environment variables NOT committed — secrets & machine-specifics …
        ▲
        └── in dev, populated from .env by DotNetEnv at startup
```

The elegant part is the last line. Production sets real environment variables; dev keeps them in the gitignored `.env` from 0.2, and a single line at the very top of `Program.cs` — before anything reads config — loads them. From the real file:

```
// Local dev: load secrets/config from the repo-root .env (the single local source of
// truth — see docs/DECISIONS.md). TraversePath walks up to find it regardless of the
// working dir; the try/catch makes it a no-op when there's no .env (e.g. production,
// which uses real environment variables).
try { DotNetEnv.Env.TraversePath().Load(); } catch { /* no .env present */ }

var builder = WebApplication.CreateBuilder(args);
```

One mechanism, two environments: to the app, dev and prod look identical — env vars either way. Nested keys use the `Section__Sub` double-underscore convention (`Jwt__Secret` in `.env` becomes `Jwt:Secret` to `IConfiguration`), which is the standard env-var transport for hierarchical config.

`appsettings.json` keeps the **non-secret** shape: section names, defaults worth committing, values a reviewer should see. It also serves as *documentation of what exists* — which the gate in §5 exploits.

3. Typed settings — validation at construction

Reading `config["Jwt:Secret"]` at call sites scatters string literals through the code, defers errors to first use, and re-parses on every read. The platform's pattern: one small class per config section, reading and validating **in its constructor**, registered as a singleton. The real `JwtSettings`:

```
// src/Api/Configuration/SettingsProvider.cs
public class JwtSettings : IJwtSettings
{
    public string SecretKey { get; }
    public string Issuer { get; }
    public int ExpiryMinutes { get; }

    public JwtSettings(IConfiguration config)
    {
        SecretKey = config["Jwt:Secret"]
            ?? throw new InvalidOperationException(
                "Jwt:Secret not configured (set Jwt__Secret in .env for dev)");
        Issuer = config["Jwt:Issuer"] ?? "Perezosoft";
        ExpiryMinutes = config.GetValue("Jwt:ExpiryMinutes", 60);

        const int MinSecretKeyLength = 32;
        if (SecretKey.Length < MinSecretKeyLength)
            throw new InvalidOperationException(
                $"Jwt:Secret must be at least {MinSecretKeyLength} characters");
    }
}
```

Read the error messages as UX, because they are: each names the key **and the fix** ("set `Jwt__Secret` in `.env` for dev"). Whoever hits this — possibly you, in six months, on a new laptop — gets handed the solution, not a scavenger hunt. And notice the *semantic* validation: not just "present" but "at least 32 characters," because an HMAC key shorter than that is a security bug that would otherwise hide until token validation mysteriously failed.

Registration happens once, in one place, and the singletons are constructed **eagerly** so a bad value kills the boot:

```
// src/Api/Configuration/SettingsRegistration.cs (abridged — real file)
public static IJwtSettings AddAppSettings(this IServiceCollection services, IConfiguration
config)
{
    // Build JwtSettings once; the same instance backs both the DI singleton and the
    // JWT-bearer TokenValidationParameters — a single source of truth.
    var jwtSettings = new JwtSettings(config);

    services.AddSingleton<IJwtSettings>(jwtSettings);
    services.AddSingleton<IRefreshTokenSettings>(new RefreshTokenSettings(config));
    services.AddSingleton<IApplicationSettings>(new ApplicationSettings(config));
    // ...
    return jwtSettings;
}
```

Two subtleties worth pausing on. The settings are constructed with `new`, *right here*, not lazily by the container — so startup **is** the validation pass; a missing key can't lurk until first resolve. And `JwtSettings` is built into a local and *returned*, because `Program.cs` needs the same values to configure JWT-bearer validation — one instance, two consumers, zero chance of drift between "the settings DI hands out" and "the settings auth actually validates with." (This is the platform's blessed shape per rule R22; .NET's `IOptions` + `ValidateOnStart` is a fine alternative — the *decision* is eager validation and one pattern everywhere, not which library performs it.)

Consumers then depend on `IJwtSettings` — an interface, so tests hand in a stub without touching `IConfiguration` at all. Config-reading is quarantined to these constructors the way MailKit is quarantined to `Infrastructure/Email`: one place, one pattern.

And your 1.2 test just broke — good. `WebApplicationFactory` boots the *real* `Program`, which now demands `Jwt:Secret` at startup; the health test dies with the very `InvalidOperationException` you wrote. This is fail-fast config working exactly as designed — the test boots the app, the app validates its config, no exceptions for test mode. The seam is the one the reference harness documents: `Program` reads config at `CreateBuilder` time, *before* the factory's config hooks run, so the value must exist as an **environment variable** that early. A static constructor on the test class is the cleanest place:

```
public class HealthEndpointTests : IClassFixture<WebApplicationFactory<Program>>
{
    static HealthEndpointTests() =>
        Environment.SetEnvironmentVariable("Jwt__Secret",
            "integration-test-jwt-secret-key-0123456789"); // ≥32 chars; value irrelevant
    // ...
}
```

The dummy only needs to *satisfy the validation* (length), because nothing in this test signs or verifies a token. When the integration harness grows up in Part 2, this seam moves into its

constructor — same idea, one shared home.

4. Defaults are decisions

Look again: `Issuer` defaults to `"Perezosoft"`, `ExpiryMinutes` to `60` — but `SecretKey` throws. Why the asymmetry? Because **a default is a decision that absence is safe**. A missing issuer name has a sensible universal answer. A missing *secret* does not — any default would be catastrophic, so absence must be fatal. When you write a settings class, sort every value into one of these bins deliberately: safe default, or refuse to boot. (Part 7 adds a third bin for optional *features*: absent config means the feature is **off** — R21's config-gated default-off. Same principle: absence maps to the safe state, never the surprising one.)

5. The gate — config that can't go undocumented

Now the structural guarantee. Nothing so far stops a developer from reading `config["Fancy:NewKnob"]` in code and telling no one — until someone deploys without it. The reference platform closes this with a test that cross-references **code against documentation**, from the real `DocAndConfigSyncTests`:

```
[Fact]
public void ConfigKeys_ReadInCode_AreDocumented()
{
    // R20: every Section:Key literal read via IConfiguration is documented — either in
    // an appsettings*.json (as a nested path) or in .env.example (as Section__Key).
    // Catches config drift where code reads a key nobody declared.
    var documented = DocumentedConfigPaths(); // parsed from appsettings*.json +
.env.example

    var readKeys = new HashSet<string>(StringComparer.Ordinal);
    var access = new Regex(
        @"(?:GetSection|GetValue<[^>]*>|GetValue|configuration|config|Configuration)\s*[\(\
[]\s*""([A-Za-z][A-Za-z0-9]*(?::[A-Za-z0-9]+)+)""");
    foreach (var f in SourceFiles(Path.Combine(RepoRoot(), "src")))
        foreach (Match m in access.Matches(File.ReadAllText(f)))
            readKeys.Add(m.Groups[1].Value);

    var undocumented = readKeys.Where(k => !documented.Any(d =>
            d.Equals(k) || d.StartsWith(k + ":") || k.StartsWith(d + ":")))
        .OrderBy(k => k).ToList();

    Assert.True(undocumented.Count == 0,
        $"Config keys read in code but not in appsettings*.json or .env.example:
{string.Join(", ", undocumented)}");
}
```

It's a source scan — grep with a build verdict. Read a key in code without adding it to `.env.example` or an `appsettings*.json`, and the suite goes red with the key's name in the failure message. (One adaptation when you write `RepoRoot()`: anchor the upward directory walk on a file your repo actually has at its root — `global.json` works perfectly and 0.2 guaranteed it exists.) The documentation *cannot* drift from the code, because drift fails CI. This is the third face of one idea you now hold: discovery over registration (1.3), isolation by base class (1.3), documentation by gate (here). None of them ask a human to remember anything.

Add the corresponding entries to `.env.example` as you go — from 0.2 it's the contract: every key, a comment saying what it does, required-or-default status. `Jwt__Secret=` goes in now

(empty value, comment "REQUIRED — ≥32 chars").

6. Run it — watch it fail correctly

The best test of fail-fast config is failing it on purpose:

```
# comment out Jwt__Secret in your .env, then:
dotnet run --project src/Api
# Unhandled exception: System.InvalidOperationException:
# Jwt:Secret not configured (set Jwt__Secret in .env for dev)
```

Boot refused, key named, fix stated. Restore the key; boots clean. Then run the suite — including your new doc-sync test — and prove the gate: read a fake key (`config["Nope:Missing"]`) anywhere in `src/`, watch the test name it, delete the line.

```
dotnet test tests/Api.Tests # all green, including ConfigKeys_ReadInCode_AreDocumented
```

7. Architecture Decision

ARCHITECTURE DECISION

The fork: how does configuration reach the code? (a) `config["key"]` at call sites; (b) .NET's options pattern (`IOptions<T>` + `BindConfiguration` + `ValidateOnStart`); (c) hand-rolled typed settings classes validating in their constructors, registered eagerly as singletons.

Chosen: (c) as this platform's single blessed pattern (R22), with ADR-001's `.env` as the dev secret source and a doc-sync test (R20) making documentation mandatory. Constructor validation gives fail-at-boot with zero framework ceremony, interfaces make consumers trivially testable, and *one pattern everywhere* means a new reader learns it once.

Rejected: (a) scatters literals, defers failure to first use, and is exactly what the R20 gate exists to catch. (b) is genuinely fine — `ValidateOnStart` achieves the same fail-fast — but brings `IOptions<T>` indirection at every consumer and *two* ways to do one thing once any hand-rolled class exists. Uniformity won. The pattern choice matters less than the properties: eager, validated, typed, documented.

The trade: hand-rolled means we own the validation code, and each new section is a small class rather than one `Bind` call. Accepted for a platform this size; revisit if settings classes multiply into the dozens.

8. Checkpoint

```
git add -A && git commit -m "feat: typed validated config — .env loading, fail-fast
settings, doc-sync gate"
git tag lesson-1.4
```

You should now be able to: draw the config layering and say where secrets live in dev vs prod; explain why validation belongs in the constructor and registration must be eager; sort a config value into "safe default" vs "refuse to boot" and defend the sort; and describe how the doc-sync gate turns documentation from a request into an invariant.

Next: *Lesson 1.5 — The error envelope* — deciding, once, what every error in the system looks like from the outside.

Lesson 1.5 — The error envelope

> **Part 1 · The walking skeleton.** The shortest lesson in the course, on purpose — the artifact is five lines. But it's a five-line *treaty*: a decision about what every error in the system looks like from the outside, made once, before there are any errors to regret. Small decisions made early are cheap; the same decisions retrofitted across forty endpoints are a migration project.

Goal: one shared error shape for the whole API; a written rule that exception internals never cross the boundary; and clarity on what an error *code* is for versus an error *message*.

Concepts & principles: the error envelope; machine-readable vs human-readable error channels; information leakage through error text; deciding API contracts before the API grows.

Maps to: R16, R18 · repo: `src/Api/Services/ErrorResponse.cs`, its uses across every controller.

Prerequisites: 1.2 (an API exists to have errors).

---

1. Motivate — the anatomy of a leaked stack trace

Two failure modes, one root cause.

Failure one: the inconsistent zoo. Without a decision, each endpoint invents its own error shape: `{ "error": "..." }` here, `{ "message": "..." }` there, `{ "errors": [ ... ] }` from whoever wrote validation, a bare string from the intern's endpoint, HTML from the framework when nobody handled it at all. Every client — your own Blazor app first of all — grows a thicket of "well, if the body has *this* shape…" parsing. The API's error surface becomes something you *reverse-engineer* rather than read.

Failure two: the confession. Somewhere, a handler does the natural thing:

```
catch (Exception ex)
{
    return StatusCode(500, new { error = ex.Message }); // ← the natural thing. Don't.
}
```

Now consider what `ex.Message` can contain, on a bad day: `"28P01: password authentication failed for user \"app_rt\""` — a Postgres error naming an internal role. `"Connection refused (10.0.3.17:5432)"` — internal topology. `"The token 'eyJhbGciOi…' is malformed"` — a credential fragment. An attacker probing your API doesn't need a vulnerability report; your exceptions *are* one, streamed on demand. This is why rule R16 exists and is checked in review: **client-facing error bodies never contain raw exception text; detail stays in server logs**, where you have access control and the attacker doesn't.

Both failures have the same fix: decide the shape once, make the safe thing the easy thing.

2. Green — the envelope

The reference platform's entire `ErrorResponse.cs`:

```
namespace Perezosoft.Api.Services;

/// <summary>The standard API error envelope. Serializes to
/// <c>{ "error": ..., "message": ... }</c> (camelCase) — the shape every error
/// response uses.</summary>
public record ErrorResponse(string Error, string Message);
```

That's the artifact. Its power is entirely in the *convention it anchors*. Usage, from the real `AuthController`:

```
return Unauthorized(new ErrorResponse("no_refresh_token", "Refresh token not found"));
return Unauthorized(new ErrorResponse("invalid_refresh_token", "Refresh token is invalid or
expired"));
return StatusCode(500, new ErrorResponse("refresh_failed", "Failed to refresh token"));
```

Two fields, two audiences, and the discipline is in keeping them apart:

`error` is for machines. A stable, snake_case code — `invalid_refresh_token`, `seat_limit_reached` — that clients branch on. It's *API contract*: renaming one is a breaking change, exactly like renaming a JSON property. When the Blazor client (3.4) decides whether to bounce a user to login or show "try again," it switches on this field, never on prose.

`message` is for humans. A safe, self-contained sentence for logs and debugging. It describes the *category* of failure in words you chose — never words an exception chose. Note what `"Failed to refresh token"` doesn't say: no exception type, no stack frame, no connection string. The interesting detail went to the server log at the `catch` site, correlated by the request's trace ID (which Lesson 4.5 makes automatic).

Why not ProblemDetails (RFC 7807), the standards-blessed answer? It's a reasonable fork — see the decision box. The short version: the envelope predates it here, is smaller, and the *rule* (one shape, no leaks) matters far more than which shape.

3. Harden — a shape is only a shape if it's the only one

A convention with exceptions is a suggestion. Two enforcement layers make this one hold:

Status codes carry the first bit of meaning. The envelope rides on top of correct HTTP semantics, and this platform is unusually disciplined about three of them — you'll build each: **401** "we don't know who you are" (2.1), **403** "we know you, your *role* may not do this" (6.1, `RequirePermission`), **402** "we know you, your *plan* doesn't include this" (5.1, `RequireEntitlement`). Three different problems, three different client reactions (sign in / hide the button / show the upgrade page), distinguishable before reading a byte of body. The envelope then says *which* permission, *which* limit.

A scan for defectors. Rule R18: all error responses use the shared helper — no anonymous `{ error = ... }` objects. It's enforced the same way as 1.4's config gate: a source scan that fails when an anonymous error shape appears in a controller. You now have three of these grep-with-a-verdict tests (MailKit containment, config sync, error shape) — cheap, blunt, effective. When you write your own (3.3 formalizes the habit into the architecture-test project), keep the pattern: scan for the *forbidden* form, fail with the file name and the fix.

One honest caveat: model-binding failures (malformed JSON, missing required fields) produce ASP.NET's built-in `ValidationProblemDetails` before your code runs. The platform

leaves that be — framework-generated 400s are consistent *with each other*, and bending the framework's validation pipeline into the envelope isn't worth the ceremony. The envelope rule governs *your* error paths. Record the exception to the rule as part of the rule; an undocumented exception is how conventions rot.

4. Run it

```
curl -k https://localhost:7160/api/nope
# 404 — and once real endpoints exist, error bodies all read:
# { "error": "not_found", "message": "..." }
```

The real payoff is invisible today. It arrives every time this course adds an endpoint and there is exactly one way an error can look — and it arrives hardest in 3.4, when the client's error handling is a single small function instead of a parser museum.

5. Architecture Decision

ARCHITECTURE DECISION

The fork: what do API errors look like? (a) Whatever each endpoint returns; (b) RFC 7807 ProblemDetails everywhere; (c) a minimal shared envelope (`{ error, message }`) + correct status codes, with framework validation left native.

Chosen: (c). The binding decisions are *one shape* and *no exception text* (R16, R18); a two-field record delivers both with zero ceremony, and the machine-readable `error` code gives clients a stable contract the status code alone can't.

Rejected: (a) isn't a decision, it's an absence of one — and it's what naturally happens if this lesson doesn't exist. (b) is genuinely respectable: standard, extensible, tooling-friendly. It lost on weight — `type` URIs and extension members nobody would populate — and on the platform's preference for the smallest thing that holds the rule. Migrating later is mechanical precisely because every error already flows through one type.

The trade: a custom shape means API consumers read *your* docs, not an RFC. Accepted for a platform whose first-party clients dominate; the public API surface (7.3) documents the envelope in its OpenAPI schema.

6. Checkpoint

```
git add -A && git commit -m "feat: shared error envelope — one shape, no exception leakage"
git tag lesson-1.5
```

You should now be able to: name the two audiences of an error and which field serves each; explain why `ex.Message` in a response body is a security defect, not a style issue; distinguish 401/402/403 and say who reacts to each; and defend deciding contract shapes *before* the surface grows.

Next: *Lesson 1.6 — CI from commit one* — everything this part built (locked restore, warnings-as-errors, the gates) starts running on every push, forever.

Lesson 1.6 — CI from commit one

> **Part 1 · The walking skeleton.** The final lesson of the skeleton, and the one that makes everything before it *permanent*. So far, the gates you've built — locked restore, warnings-as-errors, the doc-sync scan — run when you remember to run them. This lesson puts them on a machine that never forgets: a pipeline that executes on every push, on a clean runner, with no local state to flatter you. The pipeline is born small and — like the architecture-test suite it partners with — **grows a job per part** for the rest of the course.

Goal: a GitHub Actions workflow that restores in locked mode, builds with warnings-as-errors, runs the tests (Testcontainers included), scans for secrets and copyleft licenses — red on any violation, on every push and pull request.

Concepts & principles: CI as a growing gate; clean-room verification ("if it only works on your machine, it doesn't work"); supply-chain gates in the pipeline; the PR checklist as process.

Maps to: R26, R27 (+ every earlier rule, now enforced remotely) · repo: `.github/workflows/ci.yml` (jobs `build-test`, `secret-scan`, `license-scan`), `.github/forbidden-licenses.json`, `.github/pull_request_template.md`.

Prerequisites: 1.1–1.5 (there must be something worth gating); a GitHub repo to push to.

---

1. Motivate — a gate you add at the end is a gate you never designed for

Here's what happens to teams that "add CI later." Later arrives; someone writes the workflow; it fails — of course it fails, nothing was ever built on a clean machine. The tests assume a database that isn't there, the build has 87 warnings that were always going to be errors *someday*, a connection string is hardcoded somewhere, and one package resolves differently than it did in March. Fixing all of it is now a project, so the workflow gets watered down — warnings allowed, the flaky tests skipped — and the team learns that CI is a formality. The gate exists, but the gate gates nothing.

Now run the tape the other way. The pipeline exists from commit one, when the entire system is a health endpoint and a probe entity. It's green in five minutes of work, because there's nothing to be red *about*. From then on, every lesson that adds a capability adds its check while the diff is small and the author is present. The pipeline never has a catching-up crisis, because it never fell behind. That's the whole argument, and it's the same argument as warnings-as-errors in 1.1: **adopt the standard when meeting it is free.**

There's a second, subtler payoff. Your machine has your `.env`, your Docker images, your half-remembered global tools. A green build there is a claim contaminated by local state. CI is a *clean room*: fresh runner, no `.env`, nothing but the repo and the workflow. When it's green, the claim "anyone can build this from a checkout" has actually been tested — which is, notice, the same claim Lesson 0.2 made about dev machines. CI is 0.2's reproducibility promise, verified continuously.

2. The workflow — build-test, from the real file

Create `.github/workflows/ci.yml`. This is the reference platform's actual first job, and every step is a decision you already made — now enforced remotely:

```
name: CI

on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main, develop]

jobs:
  build-test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v5

      - uses: actions/setup-dotnet@v5
        with:
          dotnet-version: "10.0.x"

      # Supply-chain: restore in LOCKED mode against the committed packages.lock.json
      # files. --locked-mode fails if the resolved graph differs from the lockfile, so a
      # dependency change can't slip in without updating the (reviewed) lockfile.
      - name: Restore (locked mode)
        run: |
          dotnet restore src/Api/Perezosoft.Api.csproj --locked-mode
          dotnet restore tests/Core.Tests/Perezosoft.Core.Tests.csproj --locked-mode
          dotnet restore tests/Api.Tests/Perezosoft.Api.Tests.csproj --locked-mode

      # Warnings-as-errors comes from Directory.Build.props — the same build policy
      # locally and here, one definition. --no-restore: reuse the locked restore above
      # (don't let the build re-restore loosely).
      - name: Build (warnings-as-errors)
        run: dotnet build src/Api/Perezosoft.Api.csproj -c Release --no-restore

      # Api.Tests spins up Postgres via Testcontainers — Docker is available on
      # ubuntu-latest, so the same tests run here as locally, unchanged.
      - name: Test (Core + Api)
        run: |
          dotnet test tests/Core.Tests/Perezosoft.Core.Tests.csproj -c Release --no-restore
          dotnet test tests/Api.Tests/Perezosoft.Api.Tests.csproj -c Release --no-restore

      # Fail the build if the EF model has drifted from the latest migration/snapshot.
      - name: No pending EF model changes
        env:
          ConnectionStrings__DefaultConnection:
"Host=localhost;Port=5432;Database=ef_designtime;Username=ci;Password=ci"
        run: |
          dotnet tool install --global dotnet-ef
          dotnet ef migrations has-pending-model-changes \
            --project src/Infrastructure/Perezosoft.Infrastructure.csproj \
            --startup-project src/Api/Perezosoft.Api.csproj
```

Walk the details, because each is a small lesson:

- **`--locked-mode` then `--no-restore`.** The restore step verifies the graph against

the lockfiles (1.1's gate, now unskippable); the build and test steps then *reuse* that restore. Without `--no-restore`, `dotnet build` would quietly re-restore in normal mode — and your

carefully locked graph would be decoration.

- **Testcontainers just works.** The GitHub runner has Docker, so the suite you built in

1.3 — real Postgres and all — runs identically here. This is the moment the Testcontainers decision pays its second dividend: no "CI database" to provision, no service-container YAML to keep in sync with the fixture.

- **The EF drift check** closes a gap you couldn't feel locally: change an entity,

forget `dotnet ef migrations add`, and everything *builds* — the model and the migrations have simply parted ways, to be discovered at the next deploy. `has-pending-model-changes` compares model to snapshot and fails loudly. Note the placeholder connection string: the check opens no DB connection, but the design-time factory (1.3) *requires the key to exist* — no fallback, remember. CI documents that contract by satisfying it explicitly.

3. Two more jobs — the supply-chain gates go remote

Secret scan. 0.2 installed gitleaks as a local backstop; policy is when it runs whether you remember or not. From the real workflow:

```
  secret-scan:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v5
        with:
          fetch-depth: 0 # full history so the scan covers every commit, not just the tip

      - name: gitleaks
        env:
          GITLEAKS_VERSION: "8.21.2"
        run: |
          curl -sSfL "https://github.com/gitleaks/gitleaks/releases/download/v${GITLEAKS_VE
RSION}/gitleaks_${GITLEAKS_VERSION}_linux_x64.tar.gz" \
            | tar -xz gitleaks
          ./gitleaks detect --no-banner --redact --config .gitleaks.toml
```

Two deliberate choices worth stealing: `fetch-depth: 0` scans **all history**, because a secret committed and then "deleted" is still a leaked secret — git remembers; and `--redact`, because a scanner that prints the secrets it finds into a CI log has just created a second leak. (It runs the pinned OSS binary directly rather than a marketplace action — one less third party in the trust chain of the job whose *purpose* is trust.)

License scan. Rule R26: no copyleft in the dependency graph. Not because copyleft is bad — because for *this* platform (a commercial SaaS template) a GPL dependency imposes obligations the product can't meet, and a transitive one arrives silently with some innocent-looking package. The job inventories every license, direct **and transitive**, and fails on a prohibited list (`.github/forbidden-licenses.json`: GPL/AGPL/LGPL/SSPL):

```
  license-scan:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v5
      - uses: actions/setup-dotnet@v5
        with:
          dotnet-version: "10.0.x"
      - name: dotnet-project-licenses (fail on copyleft)
        run: |
          dotnet restore src/Api/Perezosoft.Api.csproj
          dotnet tool install --global dotnet-project-licenses
          export PATH="$PATH:$HOME/.dotnet/tools"
          dotnet-project-licenses --input src/Api/Perezosoft.Api.csproj \
            --include-transitive --use-project-assets-json \
            --forbidden-license-types .github/forbidden-licenses.json
```

Note the shape: a **prohibited-list, not an allow-list** — MIT/Apache/BSD and friends never trip a false positive, so the gate stays quiet until it has something real to say. A gate that cries wolf gets disabled; this one is designed to be believed. It's also a *separate job*, deliberately: a license failure and a build failure are different conversations, and the red X should say which one you're having.

4. The human gate — the PR template

One more file, `.github/pull_request_template.md`, auto-loaded by GitHub into every pull request: the platform's checklist — tests written first, tenant-scoping verified, docs updated, no direct UI→DB access. It gates nothing mechanically; it's the *review script* — the conversation every change must survive, written down. The division of labor is worth stating: **machines check what machines can check** (this lesson, the arch tests), **the checklist scripts what humans must check** (rules like R16's "no exception text" — review-enforced until a scan exists). Process is a gate too; it just runs on people.

5. Run it — push and watch

```
git add .github && git commit -m "ci: build-test + secret-scan + license-scan on every
push"
git push -u origin main
# → GitHub → Actions: three jobs, all green
```

Then prove the gates gate (one minute, permanent trust): bump any package version in `Directory.Packages.props` *without* updating the lockfile, push a branch, and watch `Restore (locked mode)` fail with `NU1004`. Revert. That red X is 1.1's promise, kept by a machine that doesn't know you and doesn't take your word for anything.

Where this file goes from here: e2e journeys join in 3.6; the Docker image build in 8.2; deploy-to-staging with a post-deploy smoke, then gated production, in 8.3; native platform legs in the appendix. In the reference repo this file ends at ~800 lines and fourteen jobs — every one of which you'll have written in the lesson that gave it a reason to exist.

6. Architecture Decision

ARCHITECTURE DECISION

The fork: when does CI arrive, and how much does it check? (a) Later, once there's "something real to test"; (b) now, minimal — build only, expand someday; (c) now, with every gate the codebase already claims to honor: locked restore, warnings-as-errors, tests, EF drift, secrets, licenses.

Chosen: (c). A claimed-but-unchecked standard decays silently; every gate in this job list is one the previous five lessons *already committed to*, so enforcing it remotely costs minutes now and prevents the catch-up crisis later. Growth by accretion — a job per capability, added with the capability — keeps every addition small and owned.

Rejected: (a) is how teams end up with the watered-down formality pipeline — descoped at exactly the moment it's needed most. (b) defers the cheap-now checks to when they're expensive, which is (a) with better intentions.

The trade: every push now takes a few minutes to go green, and a red X blocks merging even when "it's fine, it's just the license tool." That friction is the product. Accepted — and never bypassed with a skip flag; a gate you can wave through is a gate in name only.

7. Checkpoint

```
git tag lesson-1.6
```

Part 1 complete. You have a walking skeleton with a spine, a nervous system, and an immune system: an API answering HTTP, real-database tests, validated config, one error shape — and a clean-room pipeline that re-verifies all of it on every push, forever.

You should now be able to: argue for CI-from-day-one against "we'll add it when there's something to test"; explain `--locked-mode` + `--no-restore` as a pair; say why the secret scan reads full history and redacts; and describe the prohibited-list design and why quiet gates are trustworthy gates.

Next: *Part 2 — Identity & tenancy.* The skeleton gets people: users, tokens, and the tenant model everything else stands on.

Part 2 — Identity & tenancy

Lesson 2.1 — Users & JWT access tokens

> **Part 2 · Identity & tenancy.** The platform meets its first real domain concept — a user — and its most consequential early decision: **we build authentication ourselves** instead of adopting ASP.NET Core Identity. That choice ripples through the next twenty lessons, so this lesson earns it properly: what a JWT actually is, what belongs in one, and why "store hashes, never tokens" is the reflex to install before any token exists.

Goal: a `User` entity persists; the API can issue a signed JWT access token carrying identity claims; a protected endpoint returns 401 without one and 200 with one.

Concepts & principles: JWT anatomy (header/payload/signature); claims as the identity contract; custom auth vs framework Identity (ADR-002); hash-only credential storage; the token generator/hasher seams.

Maps to: ADR-002 · repo: `src/Core/Entities/User.cs`, `src/Api/Services/JwtTokenService.cs`, `JwtClaims.cs`, `TokenGenerator.cs`, `TokenHasher.cs`, `src/Api/Configuration/JwtValidation.cs`, `src/Api/Controllers/AuthControllerBase.cs`, `AuthController.cs` (begins), `tests/Api.Tests/TokenServiceTests.cs`, `UserServiceTests.cs`.

Prerequisites: Part 1 complete (config validation especially — `Jwt:Secret` from 1.4 is about to become load-bearing). Delete the `Probe` entity from 1.3 now; its job is done.

---

1. Motivate — the fork in the road

Type `dotnet new webapp --auth Individual` and you get ASP.NET Core Identity: user tables, password hashing, cookie login, two-factor scaffolding — batteries included. So the burden of proof is on *not* using it. Here's the case the reference platform recorded in ADR-002, and it's worth internalizing because it's really a case about **owning your core domain**:

This platform's authentication is not a checkbox; it's a *product surface*. Passwordless magic links (2.4), OAuth with account-linking rules (2.5), a `tenant_id` claim that drives all data isolation (2.6), MFA step-up on multiple sign-in paths (6.5), native clients with a different token transport (appendix) — every one of these needs precise control of *when tokens are issued, what they carry, and how sessions extend*. Identity can be bent to each of these, but you end up fighting a framework whose defaults are cookie-centric and whose schema you don't own — and the fights compound. Meanwhile the part of Identity that's genuinely hard to hand-roll — password hashing — **we don't need**, because this platform never stores passwords at all: sign-in is passwordless or via OAuth, full stop.

Strip out passwords and "authentication" reduces to two well-understood mechanics: **mint a signed claim set** (this lesson) and **manage its renewal** (2.2). Both sit on primitives the BCL provides. That's a domain worth owning.

The honest counter-case: owning auth means owning its mistakes — rotation, reuse detection, replay windows are now *your* tests (and this course writes them). If your product's auth is a checkbox — internal tool, password login is fine, nothing custom — take Identity and

spend your innovation budget elsewhere. Architecture is choosing what to own; this platform chooses auth, deliberately.

2. The `User` — smaller than you expect

```
// src/Core/Entities/User.cs — the real entity
public class User
{
    public Guid Id { get; set; } = Guid.CreateVersion7();
    public required string Email { get; set; }

    /// <summary>Display name from the OAuth provider, refreshed each sign-in.</summary>
    public string? DisplayName { get; set; }

    /// <summary>True only when the provider asserts a verified email claim.</summary>
    public bool EmailVerified { get; set; }

    /// <summary>Preferred UI language (e.g. "en", "es"), or null to fall back to the
    /// browser/OS culture. A per-user preference that follows the user across
devices.</summary>
    public string? Locale { get; set; }

    // Tenant membership is the source of truth for which tenant a user belongs to;
    // resolve it via TenantMembership (one tenant per user). See ITenantRepository.

    public DateTimeOffset CreatedAt { get; set; }
    public DateTimeOffset UpdatedAt { get; set; }

    /// <summary>OAuth identities linked to this account (one per provider).</summary>
    public ICollection<UserLogin> Logins { get; set; } = new List<UserLogin>();
}
```

Read it for what's **missing**. No password hash — decided above. No `TenantId` — the comment points at the deliberate indirection: membership is a separate entity (2.8), so "which tenant" has one source of truth and a user's account survives leaving a household. No roles — membership owns those too. Every nullable is an ADR in miniature: `DisplayName?` because providers may not send one; `Locale?` because null means "follow the device." And `Guid.CreateVersion7()` — UUIDv7 ids are time-ordered, so Postgres b-tree inserts append instead of splattering randomly (a real index-locality win over v4), while staying globally unique and unguessable enough for URLs. Add the `IEntityTypeConfiguration<User>` (unique index on `Email` — one account per address is a *rule*, so it's a *constraint*), and a migration.

3. What a JWT actually is — sixty seconds of anatomy

A JWT is three base64url segments: `header.payload.signature`. The payload is just JSON claims — **readable by anyone who holds the token** (paste one into any decoder). The signature is an HMAC-SHA256 over the first two segments with your `Jwt:Secret`: it makes the token *tamper-evident*, not *secret*. Two consequences drive everything else:

1. **Never put secrets in claims.** Claims are identity statements, not storage.
2. **Validation is pure computation.** Any API instance can verify a token with just the signing key — no session table, no auth-server round-trip. That's why JWTs scale horizontally for free, and also their weakness: *you can't revoke pure math*. A stolen access token is valid until it expires. The mitigation is the two-token design: access tokens are **short-lived** (60

minutes here) and renewal runs through a *revocable* refresh token — which is exactly Lesson 2.2.

4. Green — the token service

The real issuer, trimmed to its spine:

```
// src/Api/Services/JwtTokenService.cs
public class JwtTokenService(IJwtSettings settings, TimeProvider clock,
                             ILogger<JwtTokenService> logger) : IJwtTokenService
{
    public string IssueAccessToken(Guid userId, string email, string provider,
        string? displayName = null, string? tenantName = null,
        string? locale = null, Guid? tenantId = null)
    {
        var key = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(settings.SecretKey));
        var credentials = new SigningCredentials(key, SecurityAlgorithms.HmacSha256);

        var claims = new List<Claim>
        {
            new(ClaimTypes.NameIdentifier, userId.ToString()),
            new(ClaimTypes.Email, email),
            new(JwtClaims.Provider, provider),
            // jti is an opaque uniqueness token, not a DB key — random Guid, not UUIDv7.
            new(JwtRegisteredClaimNames.Jti, Guid.NewGuid().ToString()),
        };
        claims.Add(new Claim(ClaimTypes.Name,
            string.IsNullOrWhiteSpace(displayName) ? email : displayName));
        if (tenantId is { } tid)
            claims.Add(new Claim(JwtClaims.TenantId, tid.ToString()));
        // ... tenant_name, locale: same optional pattern

        var token = new JwtSecurityToken(
            issuer: settings.Issuer,
            audience: settings.Issuer,
            claims: claims,
            expires: clock.GetUtcNow().UtcDateTime.AddMinutes(settings.ExpiryMinutes),
            signingCredentials: credentials);

        return new JwtSecurityTokenHandler().WriteToken(token);
    }
}
```

Details that are decisions:

- **`IJwtSettings` and `TimeProvider` injected** — 1.4's validated config and the

injectable clock (formally taught in 3.3; note it's *already* here — expiry math must be testable, so ambient `DateTime.UtcNow` never appears).

- **The claim set is the contract.** `JwtClaims` centralizes the custom names as

constants — and its doc comment notes they *mirror the client's* `AppClaims`: these strings cross the wire, so they're API surface, named once. The `tenant_id` claim is optional here and earns its keep in 2.6, where it becomes the input to all data isolation. The `jti` comment repays reading: a *uniqueness* nonce wants randomness (v4), a *database key* wants ordering (v7) — the platform uses each where it belongs.

- **Validation parameters come from one place** (`JwtValidation/settings.CreateParameters()`):

the JWT-bearer middleware and the service's own `ValidateToken` share them, so "what makes a token valid" can't fork into two subtly different answers.

Wire up JWT-bearer in `Program.cs` (`AddAuthentication().AddJwtBearer(...)` using the same `IJwtSettings` instance `AddAppSettings` returned — the single-source trick from 1.4), add `app.UseAuthentication()/UseAuthorization()`, and a first protected endpoint: `GET /api/auth/me` on the nascent `AuthController` returning the caller's claims.

5. Red first, though — the tests that drove it

TDD order restored for the record — these exist before the service does (`TokenServiceTests` in the reference repo):

```
[Fact]
public void Issued_token_roundtrips_and_carries_identity()
{
    var svc = CreateService(); // stub IJwtSettings + FakeTimeProvider
    var token = svc.IssueAccessToken(userId, "a@b.test", provider: "test");

    var principal = svc.ValidateToken(token);

    Assert.NotNull(principal);
    Assert.Equal("a@b.test", principal!.FindFirstValue(ClaimTypes.Email));
}

[Fact]
public void Expired_token_is_rejected()
{
    var clock = new FakeTimeProvider(start);
    var svc = CreateService(clock);
    var token = svc.IssueAccessToken(userId, "a@b.test", "test");

    clock.Advance(TimeSpan.FromMinutes(61)); // past ExpiryMinutes=60

    Assert.Null(svc.ValidateToken(token));
}

[Fact]
public void Tampered_token_is_rejected()
{
    var svc = CreateService();
    var token = svc.IssueAccessToken(userId, "a@b.test", "test");
    var tampered = token[..^4] + "AAAA"; // corrupt the signature tail

    Assert.Null(svc.ValidateToken(tampered));
}
```

Expiry, tamper, roundtrip — the three properties that *are* a token service. The `FakeTimeProvider` makes the expiry test deterministic and instant; that's the injected clock paying rent already. At the HTTP level, the 1.2 harness gets its first auth-shaped assertions: `/api/auth/me` → 401 bare, 200 with a minted token. (In the reference repo those live in the integration harness you'll grow in 2.6's wire tests.)

6. Harden — the generator and hasher seams

Beyond JWTs, the platform is about to need many *opaque* tokens — refresh tokens (2.2), magic-link tokens (2.4), invitations (2.9). Two five-line services, built now, set the pattern for all of them:

```
public class TokenGenerator : ITokenGenerator
{
    public string GenerateToken()
        => Convert.ToBase64String(RandomNumberGenerator.GetBytes(64)); // CSPRNG, 512 bits
}

public class TokenHasher : ITokenHasher
{
    public string HashToken(string token)
        => Convert.ToBase64String(SHA256.HashData(Encoding.UTF8.GetBytes(token)));

    public bool Verify(string rawToken, string storedHash)
    {
        var computed = SHA256.HashData(Encoding.UTF8.GetBytes(rawToken ?? string.Empty));
        byte[] stored;
        try { stored = Convert.FromBase64String(storedHash ?? string.Empty); }
        catch (FormatException) { return false; }
        // FixedTimeEquals is constant-time and returns false (not throws) on a length
        // mismatch, so a malformed stored hash is simply rejected.
        return CryptographicOperations.FixedTimeEquals(computed, stored);
    }
}
```

Three security reflexes, in fifteen lines:

- **`RandomNumberGenerator`, never `Random`.** `Random` is predictable by design; a guessable token is no token. CSPRNG or nothing, for anything security-bearing.
- **The database stores hashes, never tokens.** The rule that names the lesson: when a refresh token or magic link lands in a table, it lands as SHA-256. A leaked database then yields nothing replayable — the attacker holds hashes of credentials, not credentials. (Why plain SHA-256, when passwords demand bcrypt/argon2? Because these are 512-bit *random* values — brute-forcing the preimage is hopeless. Slow hashes exist to protect *low-entropy* secrets. Using the right hash for the entropy at hand is itself the lesson.)
- **`FixedTimeEquals`.** Naïve `==` returns early on the first differing byte; timing the difference leaks how much of a guess was right. Constant-time comparison closes the side channel — and note the *fail-closed* posture of `Verify`: malformed stored hash → `false`, not an exception.

7. Run it

```
dotnet test tests/Api.Tests # token roundtrip/expiry/tamper + 401/200 wire tests
dotnet run --project src/Api
# then, with a token minted via a test helper or the temporary dev endpoint:
curl -k https://localhost:7160/api/auth/me -H "Authorization: Bearer eyJ..."
```

Paste the token into a JWT decoder and *look* at it: your claims, in plain sight. Internalize that — everything in a JWT is public; only its integrity is protected.

8. Architecture Decision

ARCHITECTURE DECISION

The fork: adopt ASP.NET Core Identity, or issue our own JWTs + refresh tokens?

Chosen: custom (ADR-002). The platform is passwordless-or-OAuth — Identity's hardest-won feature (password storage) is dead weight, while the things this product needs precise control over (claim contents driving tenancy, multiple sign-in paths converging on one issue-point, MFA step-up, native token transport) are exactly where a framework's opinions cost most. Token mechanics without passwords are small, well-understood, and — decisive point — *fully testable by us* (expiry, tamper, rotation, reuse all have first-class tests).

Rejected: Identity — for *this* product. The trade is real: we own every auth bug, and several later lessons exist to pay that debt honestly. Also rejected: an external IdP (Auth0/Entra) — a fine choice in many orgs, but it outsources the exact seams this course exists to teach, and its per-user pricing fights a template meant to run free.

The trade: ~1,500 lines of auth code across Part 2 that Identity would have given us "free," in exchange for auth that bends to the product instead of the reverse — and zero framework fights in lessons 2.4, 2.5, 6.5, and the native appendix.

9. Checkpoint

```
git add -A && git commit -m "feat: users + JWT access tokens — custom auth foundation
(ADR-002)"
git tag lesson-2.1
```

You should now be able to: decode a JWT by eye and say which parts are secret (none) and which are protected (integrity); justify custom-auth-here without defending custom-auth-everywhere; explain hash-only storage and why *fast* SHA-256 is correct for *high-entropy* tokens; and name the three properties every token service must prove by test.

Next: *Lesson 2.2 — Rotating refresh tokens & sessions* — how a 60-minute token becomes a 30-day session without becoming a 30-day theft window.

Lesson 2.2 — Rotating refresh tokens & sessions

> **Part 2 · Identity & tenancy.** Lesson 2.1 left a deliberate tension: access tokens expire in 60 minutes (because they're unrevocable), but nobody will use a product that logs them out hourly. The classical resolution is the **refresh token** — and the interesting engineering isn't the happy path, it's the two attacker-facing properties: rotation (a refresh token works *once*) and **reuse detection** (a replayed old token doesn't just fail — it burns the thief's entire access).

Goal: `POST /api/auth/refresh` exchanges a valid refresh token for a fresh access token + a *new* refresh token; the used token is dead; replaying a dead token revokes every session the user has.

Concepts & principles: the two-token session model; rotation; reuse-as-theft-signal; the HttpOnly cookie as token transport (and each cookie attribute as a decision); server-side revocability.

Maps to: ADR-002 · repo: `src/Core/Entities/RefreshToken.cs`, `src/Api/Services/RefreshTokenService.cs`, `SessionService.cs`, `CookieService.cs`, `src/Api/Controllers/AuthController.cs` (`/refresh`, `/logout`), `tests/Api.Tests/SessionServiceTests.cs`, `CookieServiceTests.cs`.

Prerequisites: 2.1 (JWTs, `TokenGenerator`/`TokenHasher`).

---

1. Motivate — the session-length trilemma

Pick two: long sessions, revocability, stateless validation. A pure-JWT design that stretches expiry to 30 days gets long sessions and statelessness — and a stolen token that works for a month with no kill switch. Short-lived JWTs alone get revocability by expiry — and hourly logouts. The two-token model takes all three by *splitting the job*:

- The **access token** (2.1) stays short-lived and stateless — validated by signature on

every request, no database touch, horizontally free.

- The **refresh token** is long-lived (30 days) but *stateful*: an opaque random value

whose hash lives in a table, checked against that table exactly once per hour-ish, when the client exchanges it. Server-side state means server-side control: revoke the row, kill the session.

The pattern's cost is that the refresh token is now the crown jewel — steal one and you own the session for a month. So the whole design bends toward protecting *it*: it rides in an `HttpOnly` cookie JavaScript can't read, it's stored only as a hash, and — the crucial move — it's **single-use**.

2. The entity — a session record wearing a token costume

```
// src/Core/Entities/RefreshToken.cs — the real entity
public class RefreshToken
{
    public Guid Id { get; set; } = Guid.CreateVersion7();
    public Guid UserId { get; set; }
    public required string TokenHash { get; set; } // never the token itself
    public DateTimeOffset IssuedAt { get; set; }
    public DateTimeOffset ExpiresAt { get; set; }
    public bool IsRevoked { get; set; }
    public required string IssuedFromIp { get; set; }

    /// <summary>OAuth provider used at sign-in for this session; carried through
    /// rotation so refreshed JWTs keep an accurate provider claim.</summary>
    public required string Provider { get; set; }
}
```

Notice what this table *is*: the platform's session store. Each row is one device's session; `RevokeAllForUser` is "sign me out everywhere"; `IssuedFromIp` is forensics. And notice `IsRevoked` — rotated tokens are **flagged, not deleted**. That's not housekeeping laziness; it's the entire reuse-detection mechanism, as you're about to see. (A scheduled cleanup job eventually prunes long-dead rows — Lesson 4.4, and now you know why that job exists.)

3. Issuing — the seams compose

```
// src/Api/Services/RefreshTokenService.cs (core of the real file)
public async Task<IssuedRefreshToken> IssueRefreshTokenAsync(
    Guid userId, string ipAddress, string provider, CancellationToken ct = default)
{
    var rawToken = tokenGenerator.GenerateToken(); // 512-bit CSPRNG (2.1)
    var tokenHash = tokenHasher.HashToken(rawToken); // SHA-256 (2.1)
    var now = clock.GetUtcNow();

    var refreshToken = new RefreshToken
    {
        Id = Guid.CreateVersion7(),
        UserId = userId,
        TokenHash = tokenHash,
        IssuedAt = now,
        ExpiresAt = now.AddDays(settings.ExpiryDays),
        IsRevoked = false,
        IssuedFromIp = ipAddress,
        Provider = provider,
    };

    var created = await repository.CreateAsync(refreshToken, ct);
    return new IssuedRefreshToken(rawToken, created); // raw → client; hash → DB
}
```

The return type says the security model in one line: `IssuedRefreshToken(RawToken, Token)` — the raw value goes to the client and is *never seen again by the server as plaintext*; the entity holds only the hash. Its doc comment states the payoff: "a database leak cannot be used to forge cookies." The generator and hasher you built in 2.1 as ten throwaway lines are now load-bearing — that's what a seam buys.

4. The refresh endpoint — where rotation and detection live

The service classifies; the controller responds. First the classifier — a presented token is exactly one of four things:

```
public enum RefreshTokenStatus
{
    Valid, // active, unexpired — safe to rotate
    Expired, // known token past its expiry — reject (no theft signal)
    Unknown, // no such hash — reject as a plain bad token
    Reuse, // a previously-rotated token is being REPLAYED — theft signal
}

public async Task<RefreshTokenInspection> InspectRefreshTokenAsync(string rawToken, ...)
{
    var token = await repository.GetByHashAsync(tokenHasher.HashToken(rawToken), ct);
    if (token is null) return new(RefreshTokenStatus.Unknown, null);
    // A revoked row whose hash is being presented = a rotated-out token replayed → theft.
    if (token.IsRevoked) return new(RefreshTokenStatus.Reuse, token);
    if (token.ExpiresAt <= clock.GetUtcNow()) return new(RefreshTokenStatus.Expired,
token);
    return new(RefreshTokenStatus.Valid, token);
}
```

And the endpoint (abridged from the real `AuthController.Refresh`):

```
var inspection = await refreshTokenService.InspectRefreshTokenAsync(rawToken, ct);
if (inspection.Status == RefreshTokenStatus.Reuse)
{
    // Replay of a rotated-out token   assume theft: revoke every session for the user.
    // Client still gets the generic error below, so the reuse signal isn't leaked.
    await refreshTokenService.RevokeAllUserTokensAsync(inspection.Token!.UserId, ct);
    logger.LogWarning("Refresh-token reuse detected for user {UserId}; revoked all
sessions", ...);
}
if (inspection.Status != RefreshTokenStatus.Valid)
    return Unauthorized(new ErrorResponse("invalid_refresh_token", "Refresh token is
invalid or expired"));

// Rotate: revoke the used token, then issue a fresh session.
await refreshTokenService.RevokeRefreshTokenAsync(validToken.Id, ct);
var session = await sessionService.IssueAsync(user, validToken.Provider, ClientIp, native,
ct);
cookieService.SetRefreshTokenCookie(Response, session.RefreshToken, Request);
return Ok(session.Response);
```

Walk the threat model, because every line answers an attack:

Why rotate at all? If refresh tokens were reusable, a stolen one is a 30-day skeleton key and you'd never know. Rotation makes every token single-use — so after a theft, *two parties* hold single-use tokens: the attacker and the legitimate user. Whichever redeems first wins… and the *second* redemption trips `Reuse`.

Why is `Reuse` a five-alarm signal? Because in a correct client, a rotated-out token is *gone* — replaced the moment it was used. The only party that presents a dead token's hash is someone who copied it earlier: a thief (or, rarely, a client bug — which also deserves investigation). The response is deliberately maximal: revoke **all** the user's sessions. Attacker *and* user get logged out everywhere; the user re-authenticates and is safe; the attacker holds nothing. One stolen token cost the attacker their entire foothold.

Why the same generic 401 for `Reuse` and `Unknown`? Look at the comment: "the reuse signal isn't leaked." If reuse returned a distinguishable response, an attacker could *probe*: present a captured token and learn from the error whether it's been used yet — i.e., whether it's still worth redeeming, or whether their theft has been noticed. Same principle as 1.5's error envelope: what an error *doesn't say* is part of its design. The signal goes where it belongs — the audit log and a `LogWarning` — not to the caller.

Why `Expired` ≠ `Reuse`? An expired-but-never-rotated token is a user who left a laptop closed for a month — normal life, not theft. Treating it as an attack would mass- revoke sessions on every stale device. Classifying failure modes *separately, then responding differently* is the deep skill this endpoint teaches.

5. The cookie — every attribute is a decision

The refresh token travels in a cookie, and the real `CookieService` sets each attribute for an articulable reason:

```
response.Cookies.Append(RefreshTokenCookieName, token, new CookieOptions
{
    HttpOnly = true, // JS can never read it → XSS can't exfiltrate it
    Secure = request.IsHttps, // HTTPS-only transport (see the dev nuance below)
    SameSite = SameSiteMode.Lax, // sent on same-site + top-level nav; CSRF-resistant
    Path = "/api/auth", // sent ONLY to auth endpoints — nothing else needs it
    // MaxAge/Expires: matches the token's ExpiresAt
});
```

- **`HttpOnly`** is the reason the refresh token lives in a cookie at all rather than

localStorage: a script-injection (XSS) can call your API as the user *while the page is open*, but it cannot *steal the durable credential* — the browser withholds it from JavaScript entirely.

- **`Path=/api/auth`** is least-privilege for cookies: the token accompanies exactly the

three endpoints that need it (`refresh`, `logout`, sign-in) and never rides along on every API call, shrinking both attack surface and bytes-per-request. (The real file carries battle scars here — a long comment about expiring a legacy `Path=/` cookie that would otherwise *shadow* the scoped one. Cookies have sharp edges; the comments are the platform remembering a real 3-hour debug.)

- **`SameSite=Lax` + `Secure = request.IsHttps`** encode a dev/prod reality: in dev the

client (`:7008`) and API (`:7160`) are different *ports* — same-site, so `Lax` works — and `Secure` follows the actual scheme because a hardcoded `Secure=true` silently drops the cookie on any non-HTTPS leg. The comment in the real file names the failure: "silently breaks refresh." In production this whole class of problem is *deleted* rather than configured — single-origin hosting (8.1) puts client and API on one origin. That decision is being planted here, two parts early, and this cookie is why.

Notice also what the endpoint does *not* require: `[Authorize]`. Refresh runs on the cookie alone — by design, because its whole purpose is to work *after* the access token expired. (Native clients, which have no cookie jar, send the token in the body under an `X-Native-Client` header — same endpoint, different transport; the appendix picks this up.)

6. Red — the tests that pin the properties

From `SessionServiceTests`/the integration suite — the three that matter, in Gherkin first (the story format from `WAYS_OF_WORKING.md`):

```
Scenario: refresh rotates the token
  Given a signed-in user with refresh token R1
  When the client exchanges R1
  Then it receives a new access token and refresh token R2
  And exchanging R1 again fails

Scenario: replaying a rotated token burns all sessions
  Given tokens R1 (rotated out) and R2 (current) for the same user
  When an attacker presents R1
  Then the response is the same generic 401 as any bad token
  And R2 no longer works either

Scenario: expired token is rejected without the theft response
  Given a token past ExpiresAt that was never rotated
  When the client presents it
  Then the response is 401
  And the user's other sessions remain valid
```

The second scenario is the one teams skip — and it's the *entire point* of rotation. Write it with two clients: rotate via client A, replay the old token via client B, then assert client A's *new* token is also dead. `FakeTimeProvider` drives the third.

7. Run it

```
dotnet test tests/Api.Tests
# then live, via the .http scratchpad: sign in (temporary dev seed), then
# POST {{host}}/api/auth/refresh → 200, Set-Cookie with a NEW value
# POST again with the OLD cookie → 401 invalid_refresh_token (and a warning in the API log)
```

Watching the reuse warning appear in the log while the client sees only a generic 401 — that's the design, observable.

8. Architecture Decision

ARCHITECTURE DECISION

The fork: how do sessions extend? (a) Long-lived JWTs (30-day expiry, no server state); (b) server sessions for everything (session id cookie, every request hits the store); (c) short JWT + rotating single-use refresh token with reuse detection.

Chosen: (c) — the standard modern trade (ADR-002). Per-request validation stays stateless (JWT signature); the stateful check runs once per access-token lifetime, at the refresh exchange, which is also the *choke point* where revocation, rotation, and theft detection all naturally live. Reuse-response = revoke-all: maximal, cheap, and user-recoverable (they just sign in again).

Rejected: (a) — unrevocable month-long credentials; a breach response of "rotate the signing key and log out *every user on the platform*" is not a plan. (b) — a DB hit on every API call buys revocation-latency we don't need at the cost of the JWT's free horizontal scaling; reasonable elsewhere, wrong here.

The trade: more moving parts than either pure option — four token states, cookie path subtleties, a cleanup job (4.4), and the client must handle mid-flight 401s by refreshing and retrying (3.4 builds exactly that). Each part is testable, and each got its test before its code.

9. Checkpoint

```
git add -A && git commit -m "feat: rotating refresh tokens — sessions, reuse detection,
HttpOnly cookie transport"
git tag lesson-2.2
```

You should now be able to: draw the two-token model and say which token is stateless and why; explain rotation as the mechanism that *converts theft into a detectable event*; justify revoke-all-on-reuse and the deliberately indistinguishable 401; and defend every attribute on the refresh cookie, including the path scoping.

Next: *Lesson 2.3 — Email I: the `IEmailSender` seam* — before magic links can be mailed, the platform needs exactly one way to send email.

Lesson 2.3 — Email I: the `IEmailSender` seam

> **Part 2 · Identity & tenancy.** Before the next lesson can mail a magic link, the platform needs to send email — and *how* you introduce that capability is a masterclass in the seam pattern. One interface in Core; MailKit quarantined behind it so hard that a test fails if it leaks; Mailpit as the dev transport so your inbox stays a test fixture; and branded, localized templates with the one logo-embedding technique real mail clients actually render. Email is also this book's best *long-game* example: the five-line interface you write today gets decorated into crash-safety in Lesson 4.2 without touching a single call site.

Goal: `IEmailSender` exists in Core; an SMTP implementation (MailKit) lives in `Infrastructure/Email/` and nowhere else — enforced by a test; a branded test email lands in Mailpit's inbox.

Concepts & principles: the seam (swap-point interface); dependency quarantine with a containment test; SMTP dev/prod duality (Mailpit ↔ Brevo, same code); email HTML as a hostile rendering target; CID inline images; server-side localization by explicit culture.

Maps to: auth rules ("`IEmailSender` is the only way to send email") · repo: `src/Core/Abstractions/IEmailSender.cs`, `src/Infrastructure/Email/SmtpEmailSender.cs`, `SmtpSettings.cs`, `BrandedEmail.cs`, `EmailStrings.resx`, `Assets/logo.png`, `tests/Api.Tests/DocAndConfigSyncTests.cs` (`MailKit_StaysBehindInfrastructureEmail`), `tests/Api.Tests/SmtpSettingsTests.cs`.

Prerequisites: 0.2 (Mailpit is already running in your compose stack); 1.4 (options pattern).

---

1. Motivate — email is a vendor decision wearing a feature costume

"Send an email" sounds like a feature. It's actually a *vendor relationship*: dev wants a fake inbox, staging wants a free SMTP relay (Brevo), production might want SES or Postmark next year, and tests want to assert "an email was sent" without any network at all. If `new SmtpClient()` calls are scattered through your services, every one of those is a migration. If instead there is exactly **one** interface and callers know nothing else, every one of those is a config change or one new class.

That's the seam pattern, and email is its cleanest first outing because the swap pressure is real and immediate — you'll swap transports *today* (Mailpit locally, Brevo when staging arrives in 8.2) without either "side" of the seam noticing.

2. The interface — small, but every word chosen

```
// src/Core/Abstractions/IEmailSender.cs — the real file
public interface IEmailSender
{
    /// <summary>
    /// Sends an HTML email. <paramref name="inlineImages"/> are embedded in the message
    /// (multipart/related) and referenced from the HTML via <c>cid:{ContentId}</c> — the
    /// only logo-embedding approach mainstream clients (Gmail/Outlook) render reliably.
    /// <para>Throws <see cref="EmailSendException"/> if delivery to the SMTP server fails;
    /// a hung server is bounded by a per-send timeout rather than stalling the request
    /// indefinitely.</para>
    /// </summary>
    Task SendAsync(string to, string subject, string htmlBody,
        IReadOnlyList<EmailInlineImage>? inlineImages = null, CancellationToken
cancellationToken = default);
}

public sealed record EmailInlineImage(string ContentId, string FileName, byte[] Content,
string MediaType);

public sealed class EmailSendException(string message, Exception innerException)
    : Exception(message, innerException);
```

Study the *placement* first: this lives in **Core**, which references nothing — so the interface mentions no MailKit type, no `MimeMessage`, nothing SMTP-flavored. It speaks the domain's language ("send this HTML to this address, these images inline") and lets Infrastructure translate. Note also the custom `EmailSendException`: callers catch a *domain* exception, not `MailKit.Net.Smtp.SmtpCommandException` — because catching a vendor's exception type is a vendor reference in disguise, and it would quietly weld call sites to the implementation the interface exists to hide.

3. The implementation — and the sharp edges it rounds off

```
// src/Infrastructure/Email/SmtpEmailSender.cs (the real file, lightly trimmed)
public class SmtpEmailSender(IOptions<SmtpSettings> options, ILogger<SmtpEmailSender>
logger) : IEmailSender
{
    private readonly SmtpSettings _settings = options.Value;

    public async Task SendAsync(string to, string subject, string htmlBody,
        IReadOnlyList<EmailInlineImage>? inlineImages = null, CancellationToken ct =
default)
    {
        var message = new MimeMessage();
        message.From.Add(new MailboxAddress(_settings.FromName, _settings.FromAddress));
        message.To.Add(MailboxAddress.Parse(to));
        message.Subject = subject;

        var builder = new BodyBuilder { HtmlBody = htmlBody };
        if (inlineImages is not null)
            foreach (var image in inlineImages)
            {
                var resource = builder.LinkedResources.Add(
                    image.FileName, image.Content, ContentType.Parse(image.MediaType));
                resource.ContentId = image.ContentId; // referenced from HTML as
cid:{ContentId}
            }
        message.Body = builder.ToMessageBody();

        using var client = new SmtpClient
        {
            // Bound a hung server: without this, a stuck Connect/Send stalls the request
forever.
            Timeout = _settings.TimeoutSeconds * 1000,
            CheckCertificateRevocation = _settings.CheckCertificateRevocation,
        };
        try
        {
            await client.ConnectAsync(_settings.Host, _settings.Port,
SecureSocketOptions.Auto, ct);
            if (!string.IsNullOrEmpty(_settings.Username))
                await client.AuthenticateAsync(_settings.Username, _settings.Password ??
string.Empty, ct);
            await client.SendAsync(message, ct);
            await client.DisconnectAsync(true, ct);
        }
        catch (OperationCanceledException)
        {
            throw; // genuine cancellation — let it propagate, don't mask as a send failure
        }
        catch (Exception ex)
        {
            logger.LogError(ex, "SMTP send to {Host}:{Port} failed", _settings.Host,
_settings.Port);
            throw new EmailSendException($"Failed to send email via
{_settings.Host}:{_settings.Port}.", ex);
        }
    }
}
```

Three details that separate production code from sample code:

- **The timeout.** SMTP servers hang; without a bound, the *user's sign-in request*

hangs with them. The `Timeout` line converts "infinite stall" into "bounded failure" — and its value is config (`SmtpSettings.TimeoutSeconds`), because 1.4 taught us where numbers like this live. (`CheckCertificateRevocation` is likewise a knob — added when a corporate network's blocked CRL endpoint made every send fail; a real scar with a real test, `SmtpSettingsTests`.)

- **The `OperationCanceledException` pass-through.** A cancelled request isn't a failed

email; catching it into `EmailSendException` would lie to callers and pollute logs with phantom SMTP errors. Distinguishing *cancellation* from *failure* is a habit worth stealing for every I/O wrapper you ever write.

- **Auth only when configured.** Mailpit takes no credentials; Brevo requires them. One

`if` makes the same code serve both — the dev/prod duality lives in config, not code.

Add `SmtpSettings` (host/port default to Mailpit — `localhost:1025` — so a fresh checkout sends to the trap with *zero* config), the `.env.example` block for a real provider (commented out, per the contract), and register in DI: `services.AddScoped<IEmailSender, SmtpEmailSender>()`.

4. The containment test — a seam with a fence

An abstraction "everyone should use" lasts until the first hurried Tuesday. The platform makes the quarantine mechanical (from the real `DocAndConfigSyncTests`):

```
[Fact]
public void MailKit_StaysBehindInfrastructureEmail()
{
    // The IEmailSender abstraction is the only way to send email; the concrete
    // MailKit/MimeKit dependency must never leak outside src/Infrastructure/Email/.
    var offenders = SourceFiles(Path.Combine(RepoRoot(), "src"))
        .Where(f => !f.Contains(Path.Combine("Infrastructure", "Email")))
        .Where(f => File.ReadAllText(f) is var t && (t.Contains("using MailKit")
                    || t.Contains("using MimeKit") || t.Contains("MailKit.") ||
t.Contains("MimeKit.")))
        .Select(Path.GetFileName).ToList();

    Assert.True(offenders.Count == 0,
        $"MailKit/MimeKit must stay behind IEmailSender: {string.Join(", ", offenders)}");
}
```

You've now built this move three times (config sync, error shapes, and this) — but note what's different here: this is the first time a *dependency* is fenced rather than a practice. The rule "only Infrastructure/Email knows MailKit exists" stops being a convention the moment this test exists; it's a property of the codebase. When 4.2 wraps this sender in an outbox decorator, and when a future maintainer swaps MailKit for something else entirely, the blast radius is one folder — *provably*.

5. Branded, localized — email HTML is a hostile country

A magic-link email is often the *first* thing your product ever shows a user — it should look like the product. `BrandedEmail` builds the HTML, and its doc comment is a compact field guide to why email templates can't just reuse your web CSS:

> Email HTML is its own world — table-based layout, inline styles, web-safe fonts only — so this does NOT reuse the app's CSS. The logo is embedded via CID (multipart/related), the one approach Gmail and Outlook render reliably (they block data-URI images).

Unpack the two traps it names. **Layout:** mail clients render HTML with engines that are years behind browsers and aggressively sanitized — no external stylesheets, patchy flexbox, `<style>` blocks stripped by some clients. Table layout + inline styles isn't retro taste; it's the *only* dialect with universal support. **The logo:** you can't hotlink it (blocked-by-default remote images) and you can't data-URI it (Gmail/Outlook block those too). The working answer is **CID embedding**: the PNG travels *inside* the message as a `multipart/related` part with a `Content-ID`, and the HTML references `cid:perezosoft-logo`. That's why `IEmailSender` has the `inlineImages` parameter — the seam's shape was dictated by a Gmail rendering rule, which is exactly the kind of knowledge that belongs *behind* a seam.

And localization — note the design, because it differs from UI localization in one important way:

```
private static readonly ResourceManager Rm =
    new("Perezosoft.Infrastructure.Email.EmailStrings", typeof(BrandedEmail).Assembly);

public static CultureInfo ResolveCulture(string? locale)
{
    if (string.IsNullOrWhiteSpace(locale)) return DefaultCulture; // "en"
    try { return CultureInfo.GetCultureInfo(locale.Trim()); }
    catch (CultureNotFoundException) { return DefaultCulture; } // fail soft, not 500
}
```

Emails are rendered **server-side, in an explicit culture** — the requester's UI language for an OTP, the *inviter's* saved locale for an invitation — so the code keys `ResourceManager` by a passed-in culture and never trusts the ambient thread culture (which on a server belongs to nobody in particular). `EmailStrings.resx` + `EmailStrings.es.resx` hold the strings; Lesson 3.5 adds the UI's resx pipeline and they'll rhyme. Also file away the *rebrand trap* the platform's docs call out: this inline logo and palette are brand touchpoints that live nowhere near the UI folder — lesson 9.1's checklist exists because everyone forgets them.

6. Red → Green — Mailpit is your assertable inbox

The test story for email is two-layered. Unit level: a `FakeEmailSender` recording `(to, subject, body)` — services in later lessons assert against it. Integration level: actually send, and *read the result back out of Mailpit* — it exposes a REST API over its inbox (`GET /api/v1/messages`), which is precisely why 0.2 chose it:

```
[Fact]
public async Task Branded_email_arrives_with_inline_logo()
{
    var sender = CreateRealSender(); // pointed at localhost:1025
    var body = BrandedEmail.MagicLink("https://example.test/x",
BrandedEmail.ResolveCulture("es"));

    await sender.SendAsync("probe@test.local", body.Subject, body.Html, body.InlineImages);

    var msg = await Mailpit.LatestMessageTo("probe@test.local"); // Mailpit's REST API
    Assert.Contains("cid:", body.Html);
    Assert.Equal(body.Subject, msg.Subject); // localized subject
}
```

In 3.6 this same trick graduates to E2E: Playwright journeys will fish OTP codes out of Mailpit to complete real sign-ins, browser-to-inbox-to-browser.

7. Run it

```
docker compose up -d # Mailpit already in your stack from 0.2
dotnet test tests/Api.Tests # containment + settings + send tests
# then send one for real (temporary dev endpoint or test), and open:
# http://localhost:8025 → your branded email, logo rendered, in Spanish if you asked
```

Seeing the actual rendered email in Mailpit's UI is worth thirty assertions: fonts, layout, logo — this is what your user's first impression looks like.

8. Architecture Decision

ARCHITECTURE DECISION

The fork: how does the platform send email? (a) SMTP client calls inline where needed; (b) a vendor SDK (SendGrid/SES) used directly; (c) one Core seam (`IEmailSender`), an SMTP implementation behind it, containment enforced by test.

Chosen: (c), with **SMTP as the protocol** rather than any vendor's API — SMTP is the one interface every provider (Mailpit, Brevo, SES, Postmark) speaks, so even the *implementation* is vendor-portable; switching providers is a config change (8.2 does exactly this for staging). The seam keeps callers domain-pure and gives 4.2 its decoration point.

Rejected: (a) welds a vendor and its failure modes into every call site — the anti-seam. (b) buys nicer delivery analytics at the price of a per-vendor rewrite and mocks that mimic a vendor SDK; if those analytics are ever needed, they arrive as *another implementation of the same seam*.

The trade: SMTP is lowest-common-denominator — no template hosting, no delivery webhooks (bounces land in a provider dashboard, not your code). Accepted; a platform can graduate individual needs later without touching a single caller.

9. Checkpoint

```
git add -A && git commit -m "feat: email seam — IEmailSender, MailKit quarantined, branded
localized templates"
git tag lesson-2.3
```

You should now be able to: place an interface on the correct side of the dependency rule and say why the exception type matters as much as the method signature; explain CID embedding and why email HTML can't share the app's CSS; state why emails localize by explicit culture; and write a containment test for any dependency you ever decide to quarantine.

Next: *Lesson 2.4 — Passwordless: magic link + OTP* — the seam gets its first real customer, and the platform gets its first sign-in.

Lesson 2.4 — Passwordless: magic link + OTP

> **Part 2 · Identity & tenancy.** The platform gets its first sign-in — and it's not a password form. Two passwordless credentials, one entity: the **magic link** (a long random token in an emailed URL) and the **OTP** (a short numeric code you type). The magic link is easy. The OTP is where this lesson earns its place, because a 6-digit code has a million possibilities and an attacker has a keyboard — the interesting engineering is entirely in the *brute-force arithmetic* and the lockout design that answers it, including one genuinely subtle bug ("resend-proof lockout") the platform's audit caught.

Goal: a user with nothing but an email address can sign in twice over — click a mailed link, or type a mailed code — and both credentials are single-use, time-limited, stored only as hashes, and brute-force-proof.

Concepts & principles: possession-based auth (email as the factor); high- vs low-entropy credentials and their different defenses; cumulative lockout windows; enumeration-resistant issuance *and* error mapping; account creation at redemption.

Maps to: ADR-C15, auth rules (`LoginToken` + `PasswordlessService`, lifetimes in config) · repo: `src/Core/Entities/LoginToken.cs`, `src/Api/Services/PasswordlessService.cs`, `AuthController` issue/redeem endpoints, `tests/Api.Tests/PasswordlessServiceTests.cs`, `OtpErrorMappingTests.cs`, `tests/Core.Tests/LoginTokenTests.cs`.

Prerequisites: 2.1 (generator/hasher, JWT issue), 2.2 (sessions to mint on success), 2.3 (email to deliver the credentials).

---

1. Motivate — passwords are a liability you can decline

Every password your system stores is a promise: to hash it with an expensive KDF, to survive credential-stuffing from every other site's breach, to run "forgot password" flows that are themselves attack surface, to nag about rotation. Decline the deal and the liability evaporates: **prove control of the inbox instead**. Anyone who can read `ana@example.test`'s email *is* — for this product's threat model — Ana; email is already the recovery root for password systems anyway, so passwordless just removes the middleman.

Two ergonomic forms of the same proof:

	Magic link	OTP
Credential	512-bit random token in a URL	6 random digits
UX	one click, same device	type a code — survives cross-device (read on phone, type on desktop)
Entropy	~10^{77} possibilities	**10^6 possibilities**
Defense	its own entropy	*everything else in this lesson*

That entropy row is the lesson's spine. Nobody brute-forces the link. The code, though — a million possibilities, five tries a code, and unlimited codes on request? Do the math before writing the code; we'll do it in §4.

2. One entity for both — `LoginToken`

```
// src/Core/Entities/LoginToken.cs — the real entity
public class LoginToken
{
    public Guid Id { get; set; } = Guid.CreateVersion7();

    /// <summary>Normalized (lower-cased) email the credential was issued to.</summary>
    public required string Email { get; set; }

    /// <summary>SHA-256 hash of the raw token/code.</summary>
    public required string CodeHash { get; set; }

    /// <summary><see cref="LoginTokenPurpose"/> — magic link or OTP.</summary>
    public required string Purpose { get; set; }

    public DateTimeOffset ExpiresAt { get; set; }

    /// <summary>Set when the credential is redeemed (or locked out); null while
usable.</summary>
    public DateTimeOffset? ConsumedAt { get; set; }

    /// <summary>Failed verification attempts — used to lock out OTP
brute-forcing.</summary>
    public int AttemptCount { get; set; }

    public DateTimeOffset CreatedAt { get; set; }

    // Derived, never stored. The *At(now) overloads are the deterministic core (testable
    // with an explicit clock); the parameterless properties are ambient-time conveniences.
    public bool IsConsumed => ConsumedAt.HasValue;
    public bool IsExpiredAt(DateTimeOffset now) => now >= ExpiresAt;
    public bool IsValidAt(DateTimeOffset now) => !IsConsumed && !IsExpiredAt(now);
}
```

Familiar decisions compounding: hash-only storage (2.1), UUIDv7 keys, lifetimes from config (1.4 — `Auth:MagicLink:TokenLifespanMinutes`, `Auth:Otp:*`). Two new ones:

- **`IsValid` is computed, never stored.** There is no `IsValid` column to forget to update — validity *derives* from `ConsumedAt` + `ExpiresAt` at the moment of asking (Golden Rule: derived values are computed, not stored as stale flags). And note the shape: `IsValidAt(now)` is the deterministic core the unit tests drive with an explicit clock; the ambient-time property is a convenience that delegates. Logic first, sugar second.
- **`ConsumedAt` is a timestamp, not a boolean.** Same storage cost, strictly more information — *when* it was redeemed is forensics the platform gets for free.

3. Issue and redeem — the magic-link half

```
// src/Api/Services/PasswordlessService.cs (real code)
public async Task<string> IssueMagicLinkTokenAsync(string email, CancellationToken ct =
default)
{
    email = Normalize(email); // trim + lower — one canonical form
    await repository.InvalidateActiveAsync(email, LoginTokenPurpose.MagicLink, ct);

    var raw = tokenGenerator.GenerateToken(); // 512-bit CSPRNG (2.1's seam)
    await repository.AddAsync(NewToken(email, LoginTokenPurpose.MagicLink, raw,
                                       settings.MagicLinkLifespanMinutes), ct);
    return raw; // → BrandedEmail → IEmailSender (2.3)
}

public async Task<User?> RedeemMagicLinkAsync(string email, string token, CancellationToken
ct = default)
{
    email = Normalize(email);
    var hash = tokenHasher.HashToken(token);
    var record = await repository.GetActiveByHashAsync(email, LoginTokenPurpose.MagicLink,
hash, ct);
    if (record is null) return null;

    record.ConsumedAt = clock.GetUtcNow(); // single-use: consumed atomically with success
    await repository.UpdateAsync(record, ct);

    return await userService.GetOrCreateByEmailAsync(email, ct); // ← account born HERE
}
```

Three quiet decisions:

- **Issue invalidates predecessors.** One active credential per email per purpose —

"request, request, request" leaves one live token, not a growing pile of valid ones.

- **The account is created at *redemption*, never issuance.** The service's doc comment

states why: "a typo'd or probed email leaves no trace." Anyone can type any address into a login box; if issuance created accounts, your Users table would fill with strangers' addresses and typos — and an attacker could *enumerate* which emails are registered by watching for "account already exists" behavior. Sign-in and sign-up collapse into one flow, and the collapse is a security feature.

- **Redemption looks up by hash** — the raw token exists only in the email and the

redeeming request, never in a query log or a table.

The controller endpoints glue this to 2.3's email and, on success, to 2.2's session issuance: redeem → `SessionService.IssueAsync` → access token + refresh cookie. The routes are worth a glance for a subtlety: `POST /api/auth/magic-link/send` issues, but verify is `GET /api/auth/magic-link/verify` — a **GET**, because it's reached by *clicking a link in an email*, not by a scripted request. (OTP, typed by hand, is symmetric POSTs: `otp/send` and `otp/verify`.) All sign-in paths in this platform converge on that same session-mint point — remember that convergence; MFA (6.5) will exploit it.

4. The OTP half — do the arithmetic first

A 6-digit code: 10^6 possibilities. Guessing has a 1-in-a-million shot per try. Sounds fine — until you multiply. Five attempts per code and free code-reissue gives an attacker who

requests a fresh code every lockout: 5 tries, resend, 5 more tries against a *new* code with a *reset* counter… The per-code counter never trips, and over hours a patient script accumulates thousands of attempts. Worse, each resend gives *this* code its own 1-in-200,000 ($5/10^6$) chance. The naive design is soft.

The platform's answer, from the real `RedeemOtpAsync` — read the comment first, it's the audit finding that shaped the code:

```
// Cumulative, resend-proof lockout: count failed attempts across EVERY code issued to
// this email in the window, not just the current one. Issuing a fresh code
// (AttemptCount=0) can't hand the attacker another budget, and the lock holds even
// against the correct code until the window elapses (CONF-5).
var windowStart = clock.GetUtcNow().AddMinutes(-settings.OtpLockoutWindowMinutes);
var failuresInWindow = await repository.CountFailedAttemptsSinceAsync(
    email, LoginTokenPurpose.Otp, windowStart, ct);
if (failuresInWindow >= settings.OtpMaxAttempts)
    return new OtpResult(OtpStatus.TooManyAttempts, null);

var record = await repository.GetLatestActiveAsync(email, LoginTokenPurpose.Otp, ct);
if (record is null)
    return new OtpResult(OtpStatus.Expired, null); // none active → expired or never issued

// Timing-safe comparison — the OTP code is low-entropy, so don't leak it via
// early-exit string equality.
if (tokenHasher.Verify(code, record.CodeHash)) { /* consume + get-or-create user */ }

record.AttemptCount++; // wrong code — count it
```

The design in three moves:

- **The budget belongs to the *email*, not the code.** Failures are counted across every

code in the window (`CountFailedAttemptsSinceAsync`), so "resend" no longer resets anything. Five failures in fifteen minutes = locked, *even for the correct code*, until the window drains. Redo the math: 5 attempts per 15 minutes = 480/day against a code that *itself expires in 10 minutes*. The attack is dead.

- **The lockout is state the attacker can't launder.** Note what the audit comment

implies about the bug it fixed: any per-code counter is attacker-resettable *by design* (they can always request a fresh code). When you design a lockout, ask: *what action resets my counter, and can the attacker perform it?*

- **Timing-safe verify, even for six digits.** Low-entropy secrets are exactly where a

timing oracle helps most — leaking "first digit right" collapses 10^6 to 10^5 fast. 2.1's `FixedTimeEquals` hasher was built for this moment.

And the numeric code generator — one line worth pausing on:

```
digits[i] = (char)('0' + RandomNumberGenerator.GetInt32(0, 10));
```

`GetInt32(0, 10)` per digit — CSPRNG *and* modulo-bias-free, versus the classic `randomBytes % 10` bug that skews digit distribution. Small crypto details are still crypto details.

5. What the caller learns — collapse the statuses

Internally there are four OTP outcomes. Externally, there are fewer — deliberately. From the real `OtpErrors`:

```
/// "No active code" (Expired) and "wrong code" (Invalid) collapse to the SAME
/// invalid_code so a caller can't probe whether an address has an outstanding OTP;
/// the full status stays server-side.
public static string ClientCode(OtpStatus status) => status switch
{
    OtpStatus.Invalid or OtpStatus.Expired => "invalid_code",
    OtpStatus.TooManyAttempts => "too_many_attempts",
    ...
};
```

Same principle as 2.2's indistinguishable 401: the error surface is *part of the attack surface*. "Does this address have a code outstanding?" is information (it reveals someone is mid-sign-in). Model statuses precisely inside; collapse them at the boundary (1.5's `error` codes); keep the distinction in logs. There's a dedicated test — `OtpErrorMappingTests` — because a *mapping* is exactly the kind of code a refactor breaks silently.

One more boundary defense wired in the controller: issuance is **rate-limited per IP** (`Auth:RateLimit:PasswordlessPermitLimit`, default 5/min) — because "email anyone a code, free, forever" is otherwise a spam cannon pointed at your SMTP reputation.

6. Red — the tests that drove it

The Gherkin scenarios (per `WAYS_OF_WORKING.md`, written before the service):

```
Scenario: magic link signs in and is single-use
  Given "ana@example.test" requested a magic link
  When she redeems it
  Then she is signed in and an account exists for her email
  And redeeming the same link again fails

Scenario: OTP lockout survives a resend
  Given an attacker has failed 5 OTP attempts for "ana@example.test"
  When a fresh code is issued and the attacker tries again — even the CORRECT code
  Then verification returns too_many_attempts until the window elapses

Scenario: expiry is enforced
  Given a magic link issued 16 minutes ago (lifespan 15)
  When it is redeemed
  Then sign-in fails
```

`PasswordlessServiceTests` drives all of these with `FakeTimeProvider` (advance past expiry; drain the lockout window) and the fake email sender from 2.3 capturing raw codes. The resend-proof scenario is the one that *distinguishes this design* — if you write one test today, write that one. In 3.6, the E2E suite re-proves the happy path end-to-end by fishing the real OTP out of Mailpit with a browser driving.

7. Run it

```
dotnet test tests/Api.Tests --filter Passwordless
dotnet run --project src/Api
# .http scratchpad: POST /api/auth/otp/send { "email": "you@test.local" }
# → open http://localhost:8025, read the branded code from Mailpit
# → POST /api/auth/otp/verify { "email": "...", "code": "123456" }
# → 200: access token + refresh cookie. You just signed in with no password in the system.
```

8. Architecture Decision

ARCHITECTURE DECISION

The fork: primary sign-in mechanism? (a) Passwords (+email reset); (b) OAuth only ("Sign in with Google, period"); (c) passwordless email credentials + OAuth as an accelerator (next lesson).

Chosen: (c) (ADR-C15). Email possession is the root of trust either way — password reset already reduces to it — so passwordless removes an entire liability class (storage, stuffing, reset flows) rather than defending it. Magic link and OTP share one entity and one service because they are one concept at two entropy levels; the low-entropy one is wrapped in cumulative lockout, rate limiting, timing-safe compare, and enumeration-resistant errors.

Rejected: (a) — maximum liability for zero differentiation; this platform would spend its auth budget defending a mechanism users increasingly don't want. (b) — excludes anyone unwilling to link a big-tech identity and makes *your* account system hostage to *their* account system; fine for internal tools, wrong default for a SaaS template.

The trade: sign-in now depends on email deliverability (SMTP down = login down — which is why 2.3 bounded that timeout, and why 4.2 will make sends crash-safe), and inbox compromise = account compromise. That last is true of password systems too — passwordless is just honest about it. MFA (6.5) exists for the users who need more.

9. Checkpoint

```
git add -A && git commit -m "feat: passwordless sign-in — magic link + OTP, cumulative
resend-proof lockout"
git tag lesson-2.4
```

You should now be able to: run the brute-force arithmetic for any short credential and size a lockout to beat it; explain why the lockout budget must attach to the email, not the code; justify account-creation-at-redemption as enumeration defense; and articulate which OTP statuses collapse at the boundary and why the *mapping itself* deserves a test.

Next: *Lesson 2.5 — OAuth & account linking* — "sign in with Google" in one line per provider, and the takeover-shaped edge case hiding inside account linking.

Lesson 2.5 — OAuth & account linking

> **Part 2 · Identity & tenancy.** "Sign in with Google" looks like a convenience feature. It's actually a *federation decision* — you're accepting identity assertions from a third party — and the dangerous part isn't the OAuth dance (the framework does that); it's the moment two identities claim the same email address. This lesson builds the one-line-per-provider registration the platform advertises, and then spends its real energy where the real risk is: the **account-linking takeover guard** and the fail-closed claim normalization behind it.

Goal: Google and Microsoft sign-in work end-to-end; adding a future provider is one constant + one `.AddXxx()` block; a same-email sign-in from a *new* provider links to the existing account — but **only** when the email is provably verified.

Concepts & principles: OAuth as identity federation (what you delegate, what you keep); provider identity vs email identity; the unverified-email takeover; fail-closed normalization (R17); trust decisions as explicit, testable objects.

Maps to: ADR-002 ("new provider = one line", takeover guard) · R17 · repo: `src/Api/Services/AuthProviders.cs`, `ClaimsExtractor.cs`, `ProviderEmailTrust.cs`, `UserService.cs` (`GetOrCreateUserAsync`), `src/Core/Entities/UserLogin.cs`, `LinkTokenService.cs`, `src/Infrastructure/ServiceCollectionExtensions.cs` (provider registration), tests: `ClaimsExtractorTests`, `ProviderEmailTrustTests`, `AuthProvidersTests`.

Prerequisites: 2.1–2.4 (all sign-in paths converge on the same session mint).

---

1. Motivate — what OAuth actually outsources

When a user clicks "Sign in with Google," you delegate *authentication* — Google proves who they are, with all its infrastructure (password vaults, device signals, its own MFA). What you do **not** outsource is *identity*: the decision "which account in *my* system is this person" remains entirely yours. That's the part frameworks can't do for you, and it's the part with the security cliff.

Here's the cliff, concretely. Ana signed up months ago via magic link as `ana@example.test`. Today someone signs in via OAuth-Provider-X, and X asserts their email is `ana@example.test`. If you match by email and link this new credential to Ana's account — congratulations, you may have just handed Ana's account to an attacker, because **some providers let users type any email into their profile, unverified**. The attacker didn't hack Ana; they hacked *your merge rule*.

Everything in this lesson exists to let the convenient thing (same email = same account, no duplicate accounts) happen *only* when it's safe.

2. The dance itself — buy, don't build

The OAuth redirect flow (authorize → callback → code exchange → claims) is standardized and hostile to hand-rolling; ASP.NET Core's authentication handlers do it properly. The platform's entire per-provider cost, from the real `ServiceCollectionExtensions`:

```
if (!string.IsNullOrEmpty(configuration["Authentication:Google:ClientId"]))
    auth.AddGoogle(google =>
    {
        google.ClientId = configuration["Authentication:Google:ClientId"]!;
        google.ClientSecret = configuration["Authentication:Google:ClientSecret"]!;
        // callback path, scopes...
    });
```

Note the guard clause: **a provider registers only when its ClientId is configured.** That's R21's config-gating instinct applied to auth — an undeployed provider isn't a broken button, it's an absent scheme. `AuthProviders.EnabledAsync` closes the loop by asking the runtime which schemes actually registered, so the login UI renders buttons for exactly the providers this deployment configured — no dead, 500-ing "Sign in with Microsoft" on an instance that never set it up. And `AuthProviders` itself:

```
public static class AuthProviders
{
    public const string Google = "google";
    public const string Microsoft = "microsoft";
    public static readonly IReadOnlyList<string> Supported = [Google, Microsoft];

    /// <summary>The authentication scheme to challenge for a provider, or null if
unsupported.</summary>
    public static string? SchemeFor(string provider) => provider.ToLowerInvariant() switch
    {
        Google => GoogleDefaults.AuthenticationScheme,
        Microsoft => MicrosoftAccountDefaults.AuthenticationScheme,
        _ => null,
    };
}
```

One place for provider names instead of string literals scattered through controllers — "new OAuth provider = one line" is really "one constant, one switch arm, one `.AddXxx()`," and nothing else in the platform changes. That's the promise; keeping it depends on the next two sections *not* being provider-specific.

The callback flow: the provider's handler materializes the external identity as a short-lived **cookie principal on a dedicated "External" scheme** — a carrier, not a session. The callback endpoint reads it, runs the identity resolution below, mints the platform's own session (2.2), and discards the carrier. External assertion in; *our* tokens out; the two never blur.

3. Normalize the claims — fail closed (R17)

Providers emit claims differently: `email_verified` as `"true"`, as `true`, or — the dangerous case — not at all. Normalization is one tiny class, and its posture is the whole point. From the real `ClaimsExtractor`:

```
public bool IsEmailVerified(ClaimsPrincipal principal)
{
    // Fail closed: verified ONLY when the provider explicitly asserts
    // email_verified="true". Absent / empty / any other value   not verified. An
    // absent claim must not be trusted — not every provider asserts it, and the
    // platform advertises "new OAuth provider = one line", so a silent
    // absent- -verified default would hand any such provider the account-takeover
    // merge path.
    var claim = principal.FindFirst("email_verified")?.Value;
    return string.Equals(claim, "true", StringComparison.OrdinalIgnoreCase);
}
```

Read the comment's chain of reasoning — it connects a *marketing promise* to a *security posture*. Because adding a provider is deliberately trivial, the system must assume some future provider will be added carelessly. If absence-of-claim defaulted to "verified," that careless one-liner would silently open the takeover path. Fail-closed normalization means the careless default is the *safe* default. This is rule R17, and it has its own unit tests (`ClaimsExtractorTests`): absent claim → false, `"True"` → true, `"1"` → false, garbage → false. Boring tests; load-bearing boundary.

4. The identity resolution — three cases, one guard

The heart of the lesson, from the real `UserService.GetOrCreateUserAsync`. A provider identity arrives as `(provider, providerUserId, email, emailVerified)` — note that **`providerUserId` is the real identity**; email is metadata that can change or lie:

```
// 1. Known provider identity → existing account
var existingUser = await repository.GetByLoginAsync(provider, providerUserId, ct);
if (existingUser != null)
    return existingUser; // the (provider, id) pair is ground truth

// 2. Same email from a new provider → same account; link the identity.
// Guard: an UNVERIFIED email claim must never attach a new credential to an
// existing account (takeover vector) — refuse outright.
var userByEmail = await repository.GetByEmailAsync(email, ct);
if (userByEmail != null && !emailVerified)
{
    logger.LogWarning("Refused unverified-email merge for {Email} via {Provider}", email,
provider);
    throw new UnverifiedEmailConflictException(email);
}
if (userByEmail != null)
{
    await repository.AddLoginAsync(new UserLogin { UserId = userByEmail.Id,
        Provider = provider, ProviderUserId = providerUserId }, ct);
    return userByEmail; // safe merge: verified email, linked
}

// 3. Brand new user — create the tenant ("household of one"), the user, and an
// owner membership linking them, atomically.
```

The three cases in threat-model terms:

- **Case 1 — the returning user.** `(provider, providerUserId)` matched a `UserLogin`

row: this exact Google account signed in before. No email logic runs at all — even if the user changed their email at Google, they get *their* account. The `UserLogin` entity (one account,

many provider identities) is what makes "Ana signs in with Google on Monday and Microsoft on Tuesday" one person instead of two accounts.

- **Case 2 — the merge, guarded.** New provider identity, known email. Convenience says

link; the guard says *only if the provider vouches for the email*. Unverified → refuse loudly (a typed exception the callback turns into a safe error page — the legitimate owner's account is untouched, and the refusal is logged as the security-relevant event it is).

- **Case 3 — the newcomer.** No matches: create user + their tenant + owner membership

in one transaction. (The "household of one" is a Part 2.8 concept arriving early — every user has a tenant from birth, which is what lets the `tenant_id` claim of 2.6 be non-optional in practice.)

One nuance sits on top: `ProviderEmailTrust`. Microsoft's v2 endpoint *omits* `email_verified` even for accounts whose email Microsoft itself verified (personal accounts). Strictly fail-closed would force those users through case-2 refusal forever. The platform's answer is a tiny, explicit trust object:

```
public bool TrustsEmailWithoutClaim(string provider) => provider.ToLowerInvariant() switch
{
    AuthProviders.Google => true, // Google always asserts the claim anyway
    AuthProviders.Microsoft => IsMicrosoftConsumersTenant(), // personal accounts only
    _ => false, // unknown/future providers stay fail-closed
};
```

Study the *shape* more than the rules: the exception to fail-closed is not an `if` buried in the merge — it's a named, injectable, unit-tested policy object (`ProviderEmailTrustTests`) whose default arm is still `false`, and whose Microsoft arm narrows to exactly the sub-case with a documented justification (consumers-tenant emails are Microsoft-verified) that *reverts automatically* if the deployment flips the configured tenant to work/school accounts. When you must carve an exception into a security rule, this is what the carving should look like: explicit, narrow, self-reverting, tested.

5. Explicit linking — the same guard from the other door

Settings will eventually offer "Connect your Google account" for an already-signed-in user (`LinkTokenService` carries the intent through the OAuth round-trip as — of course — a hashed, single-use, time-limited token; the 2.4 pattern reused). Linking *while authenticated* proves control of the platform account, so the email-match question doesn't arise — but the *other* direction still bites: the OAuth identity being attached must not already belong to another user. Same lesson, different door: every path that can attach a credential to an account needs the takeover question asked. The reference repo keeps both paths' checks in `UserService` so there's one answer.

6. Red — the tests

OAuth's redirect dance is miserable to test through a real provider — so the platform tests the *decisions* as units and the *flow* with a stand-in. The integration harness (1.2/2.6) swaps the External scheme's handler for one that authenticates from test headers — the callback then runs *its real code* against controlled identities:

```
Scenario: same verified email links, doesn't duplicate
  Given an account exists for "ana@example.test" (created via magic link)
  When a Google identity asserting email_verified=true for that email signs in
  Then no new account is created
  And the Google identity is linked to Ana's account

Scenario: unverified email must not merge (the takeover test)
  Given an account exists for "ana@example.test"
  When a provider identity with that email but NO verified claim signs in
  Then sign-in is refused with a safe error
  And Ana's account has no new login attached

Scenario: returning provider identity ignores email drift
  Given a user who previously signed in with Google id "g-123"
  When "g-123" signs in again asserting a different email
  Then they get their original account
```

The middle one is this lesson's reason to exist. If your suite has one OAuth test, make it that one.

7. Run it

Real-provider setup is config, not code: create OAuth apps in Google/Microsoft consoles, put ClientId/Secret in `.env` (the `.env.example` block from 1.4 documents the keys), run, click the button, land signed in. Then verify the *linking* behavior: sign in via OTP as `you@...`, sign out, sign in via Google with the same address — one account, two rows in `UserLogins`.

```
dotnet test tests/Api.Tests --filter "ClaimsExtractor|ProviderEmailTrust|AuthProviders"
```

8. Architecture Decision

ARCHITECTURE DECISION

The fork: how do provider identities map to accounts? (a) Separate account per provider ("you signed up with Google" forever); (b) auto-merge on email match, always; (c) merge on email match **only when verified** (explicit claim or a narrow, tested trust policy), with `(provider, providerUserId)` as the primary identity.

Chosen: (c) (ADR-002 + R17). Duplicate accounts are a genuine product failure (split data, "where are my things"), so merging matters; the guard makes it safe, and fail-closed normalization keeps it safe against the *next* provider someone adds in one careless line. `ProviderEmailTrust` handles the real-world wrinkle (Microsoft consumers) as an explicit policy object rather than a buried special case.

Rejected: (a) — punts the hard problem onto users, who end up with two half accounts and a support ticket. (b) — is the account-takeover vector, full stop; it's what several real-world breaches were made of.

The trade: legitimate users on providers that don't assert verification (outside the trust list) can't auto-merge — they land in a refusal instead of a silent link. Correct trade: the failure mode is a support case, not a takeover. The error page tells them to sign in with their original method and link explicitly from Settings.

9. Checkpoint

```
git add -A && git commit -m "feat: OAuth sign-in — one-line providers, fail-closed claims, takeover-guarded linking"
git tag lesson-2.5
```

You should now be able to: state precisely what OAuth delegates and what it can never delegate; explain why `(provider, providerUserId)` is identity and email is testimony; reproduce the takeover scenario from memory and name the two lines that prevent it; and justify when a security exception deserves to be a policy object instead of an `if`.

Next: *Lesson 2.6 — Tenancy I: the global query filter.* The `tenant_id` claim has been riding along in every token since 2.1. Time for it to earn its keep — the keystone lesson of the entire course.

Lesson 2.6 — Tenancy I: the global query filter

> **Part 2 · Identity & tenancy.** The keystone of the whole platform. Everything you build after this — billing, files, webhooks, every feature slice — leans on the guarantee installed here: *a query cannot accidentally read another tenant's data.* We earn that guarantee the hard way: build the leak first, prove it with a test, then close it structurally. The `tenant_id` claim that has ridden every JWT since 2.1 finally does its job today.

Goal: every entity marked `ITenantScoped` is filtered to the current tenant automatically, so a handler that "forgets" to scope its query still cannot read another tenant's rows — and an unauthenticated caller sees *nothing*, not *everything*.

Concepts & principles: structural vs conventional enforcement; EF Core global query filters; marker interfaces driving platform behavior; fail-closed defaults; invariant tests (pinning a platform guarantee, not a method's output).

Maps to: ADR-003 · Golden Rule 1 · R2 · repo: `src/Core/Entities/ITenantScoped.cs`, `src/Core/Abstractions/ICurrentTenant.cs`, `src/Api/Services/HttpCurrentTenant.cs`, `src/Infrastructure/Persistence/AppDbContext.cs`, tests: `TenantScopeFilterTests.cs`, `TenantInvariantTests.cs`, `HttpCurrentTenantTests.cs`, `HarnessSmokeTests.cs` (the wire-level proof).

Prerequisites: 2.1 (the `tenant_id` claim), 1.3 (the Postgres fixture — this lesson is *why* tests run against a real relational engine).

---

1. Motivate — build the leak, then look at it

Every SaaS bug that makes the news is some version of "*customer A saw customer B's data.*" The naive defense is discipline: "always add `.Where(x => x.TenantId == me)` to every query." That fails the first time a tired developer forgets — and you cannot grep your way back to confidence across a growing codebase, because absence of a `.Where` is invisible in review.

Let's feel it. Suppose a `Widget` entity and the obvious handler:

```
// first attempt — looks multi-tenant, is not
public class Widget
{
    public Guid Id { get; set; }
    public Guid TenantId { get; set; } // we store it…
    public string Name { get; set; } = "";
}

public Task<List<Widget>> ListAsync() =>
    _db.Set<Widget>().ToListAsync(); // …but nothing filters on it
```

The `TenantId` column exists, so the schema *looks* multi-tenant. Nothing enforces it. A stored column is data; isolation is a *behavior*, and no behavior has been built.

2. Red — a test that proves the leak

Seed two tenants, a widget in each, act as tenant A, and assert you see only A's rows. Against the naive handler this fails — and that failure is the whole point:

```
// tests/Api.Tests/TenantScopeFilterTests.cs (shape of the real test)
[Collection(PostgresCollection.Name)]
public class TenantScopeFilterTests(PostgresFixture fixture) : PostgresTestBase(fixture)
{
    [Fact]
    public async Task Listing_never_returns_another_tenants_rows()
    {
        var (tenantA, tenantB) = (Guid.CreateVersion7(), Guid.CreateVersion7());
        await SeedWidgetAsync(tenantA, "A's widget");
        await SeedWidgetAsync(tenantB, "B's widget");

        await using var db = Fixture.CreateContext(tenantId: tenantA); // act as A
        var visible = await db.Set<Widget>().ToListAsync(); // "forgetful" query

        Assert.Single(visible);
        Assert.Equal("A's widget", visible[0].Name); //   naive: returns BOTH
    }
}
```

Note what this test is *not*: it isn't testing a handler's logic. It's pinning an **invariant of the platform** — "a tenant-scoped read is isolated *no matter how carelessly it's written*." That's why the fix must live in the platform, not in the handler. (And note it runs on the real Postgres fixture: the EF in-memory provider's handling of global query filters diverges — a green isolation test on a fake database proves nothing. Lesson 1.3's decision was made *for this moment*.)

3. Green — make isolation structural

Three pieces. First, the marker — and the real file's doc comment deserves reading in full, because it teaches the *boundary* of the rule, not just the rule:

```
// src/Core/Entities/ITenantScoped.cs — the real file
/// <summary>
/// Marks an entity as belonging to a single tenant. Every entity implementing this is
/// automatically filtered to the current tenant by a global EF query filter (see
/// AppDbContext.OnModelCreating), so feature/domain queries are tenant-scoped by
/// default — you cannot forget to scope a read.
/// <para>
/// Platform entities that are intentionally cross-tenant or used before a tenant is
/// resolved (User, TenantMembership, auth tokens) deliberately do NOT implement this.
/// The rare cross-tenant/pre-auth lookup opts out with IgnoreQueryFilters().
/// </para>
/// </summary>
public interface ITenantScoped
{
    Guid TenantId { get; }
}
```

Second, *who is the current tenant?* A request-scoped abstraction the DbContext can depend on — and read its null contract carefully, it's the fail-closed hinge:

```
// src/Core/Abstractions/ICurrentTenant.cs — the real file
/// <summary>
/// The tenant the current request acts within, resolved from the authenticated
/// principal (the JWT tenant_id claim). This is the single, request-scoped tenancy
/// entry point: it drives the global tenant query filter in the DbContext, and
/// feature slices inject it to read their tenant id.
/// <para>TenantId is null when the caller is unauthenticated or has no tenant; the
/// global filter then matches no tenant-scoped rows (fail closed).</para>
/// </summary>
public interface ICurrentTenant
{
    Guid? TenantId { get; }
}
```

The HTTP implementation (`HttpCurrentTenant`) reads the `tenant_id` claim off the validated JWT — the claim you've been issuing since 2.1. Tests swap in the trivial settable fake from 1.3's fixture.

Third — the move that closes the leak for every entity at once, from the real `AppDbContext`:

```
// src/Infrastructure/Persistence/AppDbContext.cs (the real code)
/// <summary>Tenant the global query filter scopes to. Guid.Empty when there is no
/// current tenant — it matches no real (UUIDv7) row, so unauthenticated/tenant-less
/// callers see no tenant-scoped data (fail closed).</summary>
public Guid CurrentTenantId => _currentTenant.TenantId ?? Guid.Empty;

protected override void OnModelCreating(ModelBuilder builder)
{
    base.OnModelCreating(builder);
    builder.ApplyConfigurationsFromAssembly(typeof(AppDbContext).Assembly);

    // Tenant isolation as a structural guarantee: every ITenantScoped entity is
    // filtered to CurrentTenantId by default, so feature/domain queries can't forget
    // to scope. Discovered from the model — not hand-listed.
    foreach (var entityType in builder.Model.GetEntityTypes())
        if (typeof(ITenantScoped).IsAssignableFrom(entityType.ClrType))
            ApplyTenantFilterMethod.MakeGenericMethod(entityType.ClrType).Invoke(this,
[builder]);
}

private void ApplyTenantFilter<TEntity>(ModelBuilder builder) where TEntity : class,
ITenantScoped
    => builder.Entity<TEntity>().HasQueryFilter(e => e.TenantId == CurrentTenantId);
```

Mark `Widget : ITenantScoped`, re-run the leak test: **green**. The unchanged, still-careless `ToListAsync()` now emits `WHERE tenant_id = @currentTenant` — the handler never mentioned the tenant; the platform did. One subtlety in that lambda repays study: the filter references `CurrentTenantId` — a *property of the context* — so EF captures it as a parameter evaluated **per query**, not a constant baked at startup. One model, every request correctly scoped.

This is the discovery pattern's fourth appearance (configurations 1.3, fixture reset 1.3, config gate 1.4, now this) and its highest-stakes one: a new entity is protected the moment it implements the marker. No registration step exists to forget.

4. Harden — fail closed, then pin both properties

Look hard at one line: `_currentTenant.TenantId ?? Guid.Empty`. Why not throw? Why not skip filtering? Because tenant ids are UUIDv7 — **`Guid.Empty` matches no real row**. The failure mode of "no tenant" is therefore *see nothing*, never *see everything*. A misconfigured caller gets an empty screen, not a data breach. Pin it:

```
[Fact]
public async Task With_no_current_tenant_scoped_reads_return_nothing()
{
    await SeedWidgetAsync(Guid.CreateVersion7(), "someone's widget");
    await using var db = Fixture.CreateContext(tenantId: null); // nobody
    Assert.Empty(await db.Set<Widget>().ToListAsync()); // fail closed
}
```

Then pin the *coverage* of the rule itself. The filter protects `ITenantScoped` entities — but who checks that entities which *should* be marked, are? Rule R2's invariant test (`TenantInvariantTests` in the reference repo) walks the EF model and asserts every entity is either global-filtered or on an explicit, commented allowlist of deliberately cross-tenant types (`User`, `TenantMembership`, auth tokens, outbox plumbing). Forgetting the marker on a new entity fails the build with the entity's name — the allowlist makes the *exceptions* as reviewable as the rule.

Finally, the wire-level proof. Unit-level isolation could still be undone by a wrong registration or a middleware gap, so the integration harness (1.2) asserts the whole chain — from the real `HarnessSmokeTests`:

```
// Two owners, two tenants. Each calls the identical route with their own token; the
// JWT tenant_id claim drives the global filter, so neither can see the other's data.
var bodyA = await
_factory.CreateClientFor(a).GetFromJsonAsync<HouseholdDto>("/api/household");
var bodyB = await
_factory.CreateClientFor(b).GetFromJsonAsync<HouseholdDto>("/api/household");
Assert.DoesNotContain(bodyA.Members, m => m.UserId == b.UserId);
Assert.DoesNotContain(bodyB.Members, m => m.UserId == a.UserId);
```

Token → claim → `HttpCurrentTenant` → filter → SQL: proven at the wire, per commit.

5. Run it

```
dotnet test tests/Api.Tests --filter "TenantScope|TenantInvariant|HttpCurrentTenant"
# isolation, fail-closed, coverage invariant — the three-legged stool
```

Optional but worth it once: log the SQL (`LogTo(Console.WriteLine)`) and *see* the injected `WHERE` clause on a query whose C# has no `Where` at all.

6. Architecture Decision

ARCHITECTURE DECISION

The fork: how is tenant isolation enforced on reads? (a) Discipline — every query adds `.Where(TenantId == …)`, reviewers catch misses; (b) repository-layer scoping — data access goes through repositories that add the predicate; (c) a global EF query filter keyed on a marker interface, discovered from the model, fail-closed on no tenant.

Chosen: (c) (ADR-003). Isolation enforced by the type system + one central filter can't be forgotten per-query; discovery means new entities are protected by construction; `?? Guid.Empty` turns misconfiguration into an empty result instead of a breach. (b) arrives *as well* in 3.1 — the repository seam adds ergonomics and a sanctioned escape hatch — but the guarantee must not *depend* on everyone using the repository; defense in depth starts by not trusting your own conventions.

Rejected: (a) — it's not a mechanism, it's a hope; the news is full of its failures. Also considered: database-enforced isolation (row-level security) — *not* rejected, deferred: it arrives in 8.4 as the second wall beneath this one, once the platform is worth two walls. And schema-per-tenant / database-per-tenant: real isolation, but operationally heavy (migrations × tenants) and wrong for this platform's "thousands of small tenants" shape.

The trade: query filters are an EF feature — raw SQL escapes them (a rule for later: raw SQL is platform code, never slice code), and the filter must be *provably attached* to every entity that needs it, which is exactly what the R2 invariant test exists to hold. Write-side isolation is a separate problem — deliberately: it's the next lesson.

7. Checkpoint

```
git add -A && git commit -m "feat: tenancy reads — ITenantScoped + global query filter, fail closed (ADR-003)"
git tag lesson-2.6
```

You should now be able to: explain why a stored `TenantId` column is not multi-tenancy; add a global query filter keyed on a marker interface and say why it's discovered, not listed; defend `?? Guid.Empty` as a security decision; distinguish an invariant test from a behavior test; and name which entities deliberately live *outside* the filter and why.

Next: *Lesson 2.7 — Tenancy II: write-side.* The filter scopes what you can *read*. Nothing yet stops a buggy handler from *writing* a row stamped with someone else's tenant — or from updating cross-tenant with `ExecuteUpdate`. Isolation needs both doors.

Lesson 2.7 — Tenancy II: write-side stamping & `EnterTenant`

> **Part 2 · Identity & tenancy.** Lesson 2.6 locked the read door; this lesson locks the write door — because a filter on *reads* does nothing to stop a buggy handler from *inserting* a row stamped with someone else's tenant, or updating a row it loaded through the escape hatch. Isolation is only structural when **both directions** are. And once writes are guarded, a new question appears: how do legitimate tenant-less callers — a Stripe webhook, a future admin console — write into a tenant *without* bypassing all the guarantees? The answer is the platform's most elegant small primitive: `EnterTenant`.

Goal: a tenant-scoped entity written under the wrong tenant never reaches the database; new rows are stamped automatically; and system-authenticated operations enter a tenant scope that gives them the *same* isolation an authenticated request gets.

Concepts & principles: write-side stamping; EF `SaveChangesInterceptor`; the system-context trust level; scoping vs authorizing; the sanctioned escape hatch and its CI ban (R5); ambient context with disposable restore.

Maps to: ADR-003 (amendment) · R5, R28 · repo: `src/Infrastructure/Persistence/TenantStampingInterceptor.cs`, `src/Core/Abstractions/ITenantContext.cs`, `HttpCurrentTenant` (EnterTenant impl), tests: `TenantStampingInterceptorTests.cs`, `EnterTenantScopingTests.cs`, `ArchitectureTests.cs` (the `IgnoreQueryFilters` ban).

Prerequisites: 2.6 (the filter, `ICurrentTenant`, fail-closed `Guid.Empty`).

---

1. Motivate — the door you forgot you left open

The 2.6 test suite proves tenant A cannot *read* tenant B's rows. Write this handler, though, and watch the guarantee evaporate:

```
public async Task TransferAsync(Guid widgetId, Guid toTenantId) // hypothetical bug
{
    var widget = await _db.Set<Widget>().SingleAsync(w => w.Id == widgetId);
    widget.TenantId = toTenantId; // ← quietly re-homes A's data into B
    await _db.SaveChangesAsync(); // ← nothing objects
}
```

Or subtler: a handler builds a `new Widget { TenantId = someIdFromTheRequest }` — trusting client input for the one field that must never come from the client. Or subtler still (this one is the v2 audit's ADV-1 finding): platform code legitimately loads a row *cross-tenant* through the escape hatch, then a later code path innocently `SaveChanges`es it — an update landing under the wrong tenant with no one ever having "written" a tenant id at all.

Reads have one shape (a query) so one filter covers them. Writes have three shapes (insert, update, delete) and arrive through the change tracker — so the write-side guard lives where the change tracker converges: **`SaveChanges`**.

2. Green — the interceptor

EF Core's `SaveChangesInterceptor` sees every pending change just before it becomes SQL. The real `TenantStampingInterceptor`, core logic in full:

```
// src/Infrastructure/Persistence/TenantStampingInterceptor.cs
private static void Stamp(AppDbContext? db)
{
    if (db is null) return;

    var currentTenantId = db.CurrentTenantId;
    if (currentTenantId == Guid.Empty) return; // system/seed/cross-tenant context — not
enforced

    foreach (var entry in db.ChangeTracker.Entries<ITenantScoped>())
    {
        if (entry.State is not (EntityState.Added or EntityState.Modified or
EntityState.Deleted))
            continue;

        var tenantProperty = entry.Property(nameof(ITenantScoped.TenantId));
        var tenantId = (Guid)tenantProperty.CurrentValue!;

        // A new row may leave TenantId unset → stamp the current tenant.
        if (entry.State == EntityState.Added && tenantId == Guid.Empty)
        {
            tenantProperty.CurrentValue = currentTenantId;
            continue;
        }

        // An explicitly-tenanted insert, or any update/delete, must belong to the current
tenant.
        if (tenantId != currentTenantId)
            throw new InvalidOperationException(
                $"Refusing to {entry.State.ToString().ToLowerInvariant()} a
{entry.Entity.GetType().Name} " +
                $"for tenant {tenantId} while the current tenant is {currentTenantId}.
...");
    }
}
```

Two behaviors, both worth naming:

- **Stamping is a gift.** A handler creating an entity simply doesn't set `TenantId` —

the platform stamps it. The *correct* code is the code with *less* in it: feature authors literally cannot get this field wrong because they never touch it. (Notice the interplay with 2.6's fail-closed choice: `Guid.Empty` means "unset," which is exactly why real tenant ids being UUIDv7 matters twice.)

- **Validation is a wall.** Any insert/update/delete whose `TenantId` differs from the

current tenant throws — *before* SQL is generated. The exception message is written for the developer who hits it: what was refused, both tenant ids, and what to do instead. Wrong-tenant writes are now a *test failure during development*, not a data corruption in production.

And one registration subtlety from the real `AppDbContext` — the interceptor is attached in `OnConfiguring`, *inside the context*, not (only) in DI:

```
protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
{
    base.OnConfiguring(optionsBuilder);
    optionsBuilder.AddInterceptors(TenantStampingInterceptor.Instance, ...);
}
```

Why there? Because tests (and tools) construct contexts directly, bypassing DI — and a guard that only exists on the DI path is a guard your test double forgot. Attaching in `OnConfiguring` means **every** `AppDbContext`, however constructed, enforces the rule. The interceptor is deliberately stateless (it reads the tenant from the context being saved), so one shared `Instance` serves them all.

3. The trust boundary — who is `Guid.Empty`, really?

Look again at the early return: *no current tenant → no enforcement*. Isn't that a hole? It's a **trust level**, and drawing it consciously is this lesson's second idea.

A context with no current tenant is the *system* context: migrations, seeders, the outbox dispatcher (Part 4), tenant-dissolution teardown (2.9). System code is exactly the code that legitimately touches many tenants — and it's held to a different standard: it must be *platform* code, reviewed as such, and the read-side twin of this trust level (the `IgnoreQueryFilters()` / `QueryAllTenants()` escape hatch) is **banned from feature slices by an architecture test** (R5 — the scan you'll formalize in 3.3 fails the build if the string appears under `src/Api/Features/`). Feature code gets the guarded world with no exit; platform code gets the trusted world with review. Two tiers, machine-separated.

But there's a third kind of caller, and it's the interesting one: code that has **no JWT but a trusted tenant id**. A Stripe webhook arrives bearing a cryptographic signature (5.2) and *names* a tenant; the future admin console (7.5) impersonates into one. The lazy answer is "run them as system and be careful." The platform's answer is better — from the real `ITenantContext`:

```
// src/Core/Abstractions/ITenantContext.cs
/// Enters a tenant context for system/integration operations that have no JWT but a
/// TRUSTED tenant id — e.g. a Stripe-signature-verified webhook. While the returned
/// scope is active, ICurrentTenant.TenantId resolves to the entered tenant, so the
/// write-stamping interceptor and the global read filter scope to it. Such operations
/// therefore get the SAME structural tenant isolation an authenticated request gets,
/// instead of bypassing it with the cross-tenant escape hatch.
///
/// This primitive only SCOPES; it does not AUTHORIZE. The caller MUST have already
/// authenticated the tenant id by other means (a verified webhook signature, an admin
/// authz check).
public interface ITenantContext
{
    IDisposable EnterTenant(Guid tenantId);
}
```

Usage reads like what it is — a scoped assumption:

```
// inside the (signature-verified) billing webhook handler, Part 5:
using (tenantContext.EnterTenant(tenantIdFromVerifiedEvent))
{
    // in here, reads are filtered to this tenant and writes are stamped/validated
    // exactly as if a user of this tenant had made an authenticated request
    subscription.Status = newStatus;
    await db.SaveChangesAsync();
} // dispose restores the previous tenant (nesting supported)
```

The doc comment's sharpest sentence deserves a highlight: **"only scopes; does not authorize."** `EnterTenant` never decides *whether* you may act as tenant X — the webhook's signature check or the admin's authz check did that. It decides that, having been authorized, you operate under the *full* isolation regime rather than the trusted bypass. The alternative — webhook code running system-level with hand-rolled `WHERE` clauses — is precisely the conventional-discipline world 2.6 was built to abolish. This primitive exists so the platform never has to choose between "no JWT" and "no guardrails."

Implementation-wise, `HttpCurrentTenant` doubles as the `ITenantContext`: `EnterTenant` swaps the ambient tenant and returns an `IDisposable` that restores the previous value (nesting supported) — the same enter/restore shape you saw in the test double back in 1.3, which is no accident: the fixture's `TestCurrentTenant` implements both interfaces so services under test see one coherent ambient tenant.

4. Red — the tests that pin all three behaviors

From the real `TenantStampingInterceptorTests` / `EnterTenantScopingTests`, as Gherkin:

```
Scenario: new rows are stamped
  Given the current tenant is A
  When a Widget with unset TenantId is added and saved
  Then the persisted row's TenantId is A

Scenario: wrong-tenant writes are refused — all three states
  Given the current tenant is A and a row belonging to B (loaded via the escape hatch)
  When the row is modified / deleted / a new row is added with TenantId = B
  Then SaveChanges throws before any SQL runs
  And the message names both tenants

Scenario: EnterTenant gives a webhook the same isolation as a request
  Given no current tenant (system context)
  When the handler enters tenant A, writes, and disposes the scope
  Then the write is stamped/validated as A
  And after dispose the context is system again (and nested scopes restore correctly)
```

The middle scenario is ADV-1's regression test — the audit finding that widened the interceptor from "validate inserts" to "validate *every* state." Audits feed tests; tests hold the line.

One honest boundary the interceptor's own comment draws: **bulk operations (`ExecuteUpdate`/`ExecuteDelete`) bypass the change tracker** — no `SaveChanges`, no interceptor. The global *read* filter still constrains what they touch (they compose onto the filtered query), but the write-side wall has this known gap at the edges, held by review and by 8.4's database-level backstop. Structural enforcement doesn't mean pretending the structure covers everything; it means *knowing exactly where it ends*.

5. Run it

```
dotnet test tests/Api.Tests --filter "TenantStamping|EnterTenant"
```

Then feel the wall once: in any handler, set a `TenantId` to a random Guid and `SaveChanges`. The exception you get — naming the entity, both tenants, and the rule — is the platform teaching your future teammates in the only classroom that scales.

6. Architecture Decision

ARCHITECTURE DECISION

The fork: how are *writes* kept tenant-correct? (a) Trust handlers to set `TenantId` (review catches mistakes); (b) require every write to pass through a repository method that takes the tenant explicitly; (c) a SaveChanges interceptor that stamps unset inserts and refuses cross-tenant writes, plus `EnterTenant` for trusted tenant-less callers.

Chosen: (c) (ADR-003 amendment). The change tracker is the one choke point every tracked write crosses, so enforcement there is complete-by-construction for tracked writes; stamping makes correct code *smaller* than incorrect code; and `EnterTenant` extends the same regime to signature-authenticated work instead of granting it the bypass. Registered in `OnConfiguring` so no construction path escapes.

Rejected: (a) — the same hope 2.6 rejected for reads, with worse blast radius. (b) — helps ergonomics (and 3.1 builds it!) but can't be the *guarantee*: any code holding the DbContext can sidestep a repository, and the platform refuses to found isolation on "everyone used the blessed API."

The trade: an ambient-context primitive (`EnterTenant`) is ambient state — the classic critique applies (invisible in signatures). Contained by its shape: scoped, disposable, nest-restoring, and used only at authenticated entry points, never as a general parameter-passing dodge. And the tracked-write wall knowingly excludes bulk ops — a gap documented in code, patrolled by review, and closed for good by the RLS backstop in 8.4.

7. Checkpoint

```
git add -A && git commit -m "feat: tenancy writes — stamping interceptor + EnterTenant
primitive (ADR-003)"
git tag lesson-2.7
```

You should now be able to: name the three write shapes and where they converge; explain why stamping-plus-refusal beats "set it yourself"; articulate the system trust level and the difference between *scoping* and *authorizing*; and state precisely where the tracked-write guarantee ends and what covers the remainder.

Next: *Lesson 2.8 — Tenants & membership* — the entities behind the claim: what a tenant actually *is*, and the membership model that decides everything from "who's in my household" to "what happens when the owner leaves."

Lesson 2.8 — Tenants & membership

> **Part 2 · Identity & tenancy.** Two lessons of machinery have enforced "the current tenant" without ever defining what a tenant *is*. Time to model it — and the modeling lesson here is about restraint and invariants: the `Tenant` is four properties; all the interesting decisions live in **membership**, the link entity that answers who belongs where, in what role, and — the questions that actually bite — what happens on remove, transfer, and leave. Lifecycle transitions are where domain models earn or lose their integrity.

Goal: `Tenant` and `TenantMembership` exist with their invariants (one tenant per user, exactly one owner per tenant) enforced; the app-facing `/api/household` surface supports get/rename/remove-member/transfer/leave, each transition preserving every invariant.

Concepts & principles: link entities as decision records; invariants as constraints + result enums; the "membership, not `User.TenantId`" indirection; result types over exceptions for domain outcomes; lifecycle design ("nobody is ever tenant-less").

Maps to: ADR-003, ADR-C1/C2 · repo: `src/Core/Entities/Tenant.cs`, `TenantMembership.cs`, `src/Api/Services/TenantService.cs`, `TenantModels.cs`, `src/Api/Controllers/HouseholdController.cs`, `src/Core/Repositories/ITenantRepository.cs`, tests: `UserServiceTests` (tenant-of-one), integration `HarnessSmokeTests` (`/api/household`), later `MemberRoleManagementTests`.

Prerequisites: 2.6/2.7 (the machinery this model feeds), 2.1 (users).

---

1. Motivate — the model is the answer sheet

Product questions this lesson must answer, each of which has burned a real SaaS:

- Can a user belong to two tenants? (Your data model answers even if you never decided.)
- Ana leaves the household — does the shopping list she created leave with her?
- The owner deletes their account — is the household now headless? Orphaned? Gone?
- Two admins demote each other simultaneously — who wins?

Answer these *in entities and constraints* and the platform behaves consistently forever; answer them ad hoc per endpoint and every feature reinvents the rules until two of them disagree. So: model first, endpoints second.

2. The entities — one boring, one load-bearing

```
// src/Core/Entities/Tenant.cs — the real file, in full
public class Tenant
{
    public Guid Id { get; set; } = Guid.CreateVersion7();
    public required string Name { get; set; }
    public DateTimeOffset CreatedAt { get; set; }
    public DateTimeOffset UpdatedAt { get; set; }
}
```

Four properties. That's not underdesign — it's the discipline of putting state where it belongs: billing state will live in a `Subscription` projection (Part 5), member count is *derived* (never a stored counter that drifts — Golden Rule 4), settings arrive when a feature needs them. A tenant is an identity to hang ownership off; resist making it a junk drawer.

```
// src/Core/Entities/TenantMembership.cs — the real file
/// <summary>
/// Links a user to their tenant with a role. A user belongs to exactly one tenant at a
/// time (unique on UserId); a tenant has exactly one owner. Identity (logins, inbox)
/// stays user-scoped; app data is scoped to the tenant. Membership — not User.TenantId —
/// is the source of truth for tenant resolution.
/// </summary>
public class TenantMembership
{
    public Guid Id { get; set; } = Guid.CreateVersion7();
    public Guid TenantId { get; set; }
    public Guid UserId { get; set; }
    public string Role { get; set; } = TenantRoles.Member; // owner | admin | member
    public DateTimeOffset JoinedAt { get; set; } = DateTimeOffset.UtcNow;
}
```

The doc comment is a compressed ADR — unpack its three commitments:

"Exactly one tenant at a time (unique on UserId)." The biggest product decision in Part 2, and it's enforced as a **unique index**, not a code path. Multi-tenant-per-user (Slack-style workspace switching) is a legitimate other product — with tenant pickers, per-tenant tokens, "which tenant is this request for" on every call. This platform's domain (a household product) wants one home per user, and buys enormous simplification with it: the JWT carries *the* `tenant_id`, not a selected one. Know which product you're building; put the answer in the schema.

"Membership — not `User.TenantId` — is the source of truth." A `TenantId` column on `User` would be simpler — and would weld identity to belonging. As a separate entity, membership can carry *belonging-specific* state (`Role`, `JoinedAt`), be deleted without touching the account, and later support history. The user's account — logins, locale, notifications — survives every household transition. Identity is user-scoped; app data is tenant-scoped; membership is the hinge (ADR-C1/C2, in code).

"Exactly one owner." Not a preference — an invariant every transition below must conserve. `TenantRoles` pins the vocabulary (`owner > admin > member`, ordered — the matrix that gives the ordering teeth is RBAC's, Lesson 6.1; today only *owner* gates anything).

Neither entity implements `ITenantScoped` — they're on 2.6's documented allowlist. Membership *is* the thing tenant resolution derives from; filtering it by the thing it defines would be circular. First-hand proof the allowlist isn't a formality.

3. Where tenants come from — the household of one

No "create tenant" endpoint exists. Look back at 2.5's case 3 and 2.4's redemption: both funnel into one private `CreateUserWithTenantAsync`, which writes **user + tenant + owner membership atomically** — three rows, one `IUnitOfWork` transaction, all-or-nothing (a crash mid-way must not birth a user with no household). Every user is thus born an owner of their own tenant-of-one; there is no moment — not one request — in which an authenticated user

has no tenant. That non-existence of a "tenant-less user" state kills an entire class of null-handling and onboarding UX, and it's why 2.6's claim `tenant_id` can be relied on rather than defended against.

This "nobody is ever tenant-less" invariant now becomes the design key for every transition below.

4. Transitions — where models go to die (or not)

The service surface, from the real `TenantService` — notice it speaks in **result enums**, not exceptions:

```
public enum RemoveMemberResult { Removed, NotAMember, CannotRemoveOwner }
public enum TransferResult { Transferred, TargetNotMember, ConcurrentModification }
public enum LeaveOutcome { Left, MustTransferFirst, ConfirmationRequired, Dissolved }
```

A member being non-removable-because-owner isn't *exceptional* — it's a legal outcome the UI must explain. Exceptions are for broken invariants; enums are for domain answers. (The controller maps each to 1.5's envelope: `CannotRemoveOwner` → `{ "error": "cannot_remove_owner", ... }` — machine-readable all the way out.) Now the transitions, each read against the invariants:

Remove a member. From the real doc comment: "*lands them in a fresh tenant-of-one (owner) so they are never tenant-less. Their contributed data stays with the tenant.*" Two decisions in one sentence. Re-homing conserves the no-tenant-less invariant even under an *involuntary* transition. And the data question — the shopping list Ana created — is answered by tenancy itself: app data belongs to the *tenant* (it's `ITenantScoped`, stamped with the tenant's id, 2.7). Ana's authorship was participation, not ownership; she leaves, the list stays. The model made this decision back in 2.6 — this lesson just makes it visible. And `CannotRemoveOwner`: removal would leave the tenant headless; ownership *moves*, never vanishes.

Transfer ownership. Demote-owner + promote-target — two rows that must flip *atomically* (a crash between them = zero or two owners; a transaction makes it all-or-nothing). And note `ConcurrentModification`: transfer is guarded against the racing-admins problem with optimistic concurrency — if the membership rows changed under you, the transfer reports it instead of silently double-applying. Invariants under *concurrency*, not just under happy sequences.

Leave. The richest one — walk `LeaveOutcome`'s four arms as a decision tree: a non-owner leaves (re-homed, data stays); an owner with members can't leave until they transfer (`MustTransferFirst` — the invariant, stated as a 409); a *sole* owner leaving means destroying the tenant, which is irreversible and therefore demands an explicit confirmation round-trip (`ConfirmationRequired` → client re-sends with `confirm_dissolve: true` → `Dissolved`). That confirm-then-wipe dance previews 2.9, where "wipe" gets its real machinery.

5. The app-facing surface — and the rename seam

The controller binds it all to HTTP — `GET /api/household`, `PUT` rename, `DELETE members/{id}`, `POST transfer-ownership`, `POST leave` — with one naming oddity worth explaining rather than hiding: the route says **household**, the code says **tenant**. Deliberate bilingualism. *Tenant* is platform vocabulary — every seam, filter, and rule from 2.6/2.7 speaks it, and it stays stable

across products built from this template. *Household* is this app's word for it — user-facing, changeable per product (a team app renames it "workspace" by touching the controller layer and UI strings only; the rebrand checklist in 9.1 includes exactly this). Keep the dictionary small and one-directional: domain term inside, product term at the boundary.

Owner-only actions (rename, remove, transfer) are enforced at the controller via the membership role — full RBAC arrives in 6.1; today the check is "is caller the owner," which is the only distinction the model needs yet. The wire tests you met in 2.6's harness (`/api/household` returning *your* household, cross-tenant invisibility) now read as what they were all along: this lesson's integration suite.

6. Red — the tests

```
Scenario: removing a member re-homes them
  Given a tenant with owner Ana and member Ben, where Ben created a Widget
  When Ana removes Ben
  Then Ben is the owner of a fresh tenant-of-one
  And the Widget still belongs to Ana's tenant

Scenario: ownership transfer is atomic and conserves single-owner
  Given owner Ana and member Ben
  When Ana transfers ownership to Ben
  Then Ben is owner and Ana is admin — and at no observable point were there zero or two
owners

Scenario: sole owner leaving requires explicit confirmation
  Given Ana alone in her tenant
  When she leaves without confirmation
  Then the outcome is ConfirmationRequired and nothing changed
  When she confirms
  Then the tenant and its data are gone and Ana is re-homed
```

Each scenario is an invariant wearing a story. The membership-lifecycle E2E journey (built with the UI in Part 3/6) replays them through a real browser.

7. Run it

```
dotnet test tests/Api.Tests --filter "Tenant|Household"
# .http: GET /api/household → your tenant-of-one, role "owner", member list of you
```

8. Architecture Decision

ARCHITECTURE DECISION

The fork: how do users relate to tenants? (a) `User.TenantId` — a column; (b) many-to-many memberships (multi-workspace); (c) a membership link entity with a **unique-per-user** constraint — one tenant at a time, roles on the link.

Chosen: (c) (ADR-003). The link entity keeps identity and belonging separable (accounts survive transitions; roles/joined-at live where they belong) while the unique index keeps the *product's* answer — one home — enforced by the schema. Transitions are modeled as result enums with re-homing so no path strands a user tenant-less; ownership is conserved by construction (transfer-only, atomic).

Rejected: (a) — welds identity to belonging; every lifecycle transition turns into account surgery, and role-on-membership has nowhere to live. (b) — a different product; adopting its complexity (tenant pickers, per-request tenant selection, N×M permission surfaces) without its requirement is pure cost. The link-entity shape deliberately keeps (b) *reachable* — drop the unique index and add a picker — which is the correct way to keep an option open: pay for it only if exercised.

The trade: "one tenant per user" is a real product constraint (no guest memberships across households), and re-homing means removals *create* tenants — tenant count ≠ paying customers, which Part 5's billing must (and does) treat knowingly.

9. Checkpoint

```
git add -A && git commit -m "feat: tenants + membership — invariants, transitions,
household surface (ADR-003)"
git tag lesson-2.8
```

You should now be able to: defend membership-as-entity against the simpler column; name both schema-enforced invariants and every transition that must conserve them; explain re-homing and whose data stays where (and why the model already decided); and argue result-enums-over-exceptions for domain outcomes.

Next: *Lesson 2.9 — Invitations, dissolve & the contributor seam.* How anyone ever joins someone *else's* household — and the decentralized-teardown seam that GDPR will thank us for in Part 7.

Lesson 2.9 — Invitations, dissolve & the contributor seam

> **Part 2 · Identity & tenancy — finale.** Two jobs close out the chassis. First, invitations: the only way a tenant ever gains a second member, built entirely from parts you already own (hashed single-use tokens, branded email, membership transitions). Second — the deeper one — *dissolve*: when a tenant dies, every feature's data must die with it, including the data of **features that don't exist yet**. That impossible-sounding requirement has a beautiful answer, the `ITenantDataContributor` seam — the platform's purest example of the Open/Closed principle, and the hook GDPR will hang the entire export/erasure epic on in Part 7.

Goal: an owner can invite by email; whoever proves control of the invite token joins (with correct re-homing of *their* old tenant); a dissolving tenant's data is wiped feature-by-feature through a discovered seam — with a test that fails any future feature that forgets to register.

Concepts & principles: composing prior seams into a feature; identity-at-redemption (the invite token authenticates the *invitation*, the login authenticates the *person*); Open/Closed via DI fan-out; teardown inside one transaction; guarding solo-owner data abandonment.

Maps to: ADR-003/ADR-011 (the seam GDPR extends) · repo: `src/Core/Entities/TenantInvitation.cs`, `src/Api/Services/TenantInvitationService.cs`, `src/Api/Controllers/HouseholdInvitationsController.cs`, `src/Core/Abstractions/ITenantDataContributor.cs`, `src/Api/Services/TenantDissolutionService.cs`, tests: `TenantInvitationTests` (Core), `WipeDataTests.cs`.

Prerequisites: 2.8 (membership + transitions), 2.4 (the token pattern), 2.3 (email).

---

1. Motivate — two lifecycle holes

Hole one: every tenant so far is a household of one. There is no path by which Ana's household ever contains Ben — and "Ben types Ana's household id" is obviously not it. Joining must be *invitation-shaped*: initiated by an owner, addressed to a person, consensual on both sides, expiring, revocable.

Hole two, quieter and worse: 2.8's `Dissolved` arm claims the tenant's data gets destroyed. *Which* data? Today, a `Note` here, a `Widget` there. Next month, files, notification prefs, webhook subscriptions, billing counters — written by features that don't exist yet. A central `WipeTenant()` method listing every table is doomed to incompleteness: each new feature must remember to edit it, and the one that forgets leaves orphaned rows — data that legally *must* die (wait for GDPR) quietly surviving. The platform needs teardown that **new features join automatically**.

2. Invitations — old parts, one new idea

The entity is 2.4's pattern wearing a new purpose — hashed single-use token, expiry, computed validity — plus a status lifecycle:

```
// src/Core/Entities/TenantInvitation.cs (the real shape)
public class TenantInvitation : ITenantScoped // ← note the marker!
{
    public Guid Id { get; set; } = Guid.CreateVersion7();
    public Guid TenantId { get; set; }
    public required string InvitedEmail { get; set; } // normalized lower-case
    public Guid InvitedByUserId { get; set; }
    public string Status { get; set; } = InvitationStatuses.Pending; //
pending|accepted|revoked|expired
    public required string TokenHash { get; set; } // raw token revealed once, at creation
    public DateTimeOffset CreatedAt { get; set; }
    public DateTimeOffset ExpiresAt { get; set; }

    public bool IsValidAt(DateTimeOffset now) => Status == InvitationStatuses.Pending &&
!IsExpiredAt(now);
}
```

`ITenantScoped` — the first *feature-owned* entity to wear the marker since the machinery went in. Invitations belong to the inviting tenant: the owner lists *their* pending invites, 2.6's filter scopes them for free, 2.7 stamps them on insert. The chassis just works; this is what building *on* it feels like.

The flow composes seams end to end: owner posts an email address → service invalidates prior pending invites for it, mints token via `ITokenGenerator`, stores the hash, lifespan from config (`Auth:Invitation:LifespanDays`, 1.4) → `BrandedEmail` renders the invite **in the inviter's saved locale** (2.3's explicit-culture design paying off — the recipient sees the language of the household inviting them) → `IEmailSender` delivers.

The one genuinely new idea is at redemption, and the entity's doc comment states it precisely: "*redeemed by whoever is signed in and presents the single-use token — **identity is the login, not the bare email***." Unpack that, because it's a subtle authorization design:

- The invite link lands on `/join`. The redeemer must **sign in first** (any method —

OTP, Google, whatever *they* have). The token proves they hold the invitation; the login proves who they are. Two proofs, two different questions.

- Consequence: the invite can be *forwarded*. Ana invites `ben@work.test`; Ben redeems

while signed in as `ben@personal.test` — and joins as his real, existing account rather than growing a duplicate. The addressed email is *delivery routing*, not identity. (Compare 2.4, where email possession *is* the identity claim — same token machinery, opposite trust conclusion, both correct for their question.)

- Joining triggers 2.8's transitions: Ben's old tenant-of-one is handled by the same

leave semantics — and here the contributor seam makes its entrance early. What if Ben's solo tenant *has data*? Silently discarding it on join would be the data-abandonment bug. The platform checks `HasDataAsync` across all contributors and requires explicit confirmation — "joining will delete your current household's data" — before dissolving it. The seam serves its first customer before dissolve even ships.

Revocation (owner cancels a pending invite) and expiry round out the lifecycle; `TenantInvitationTests` in Core.Tests drive the validity state machine with explicit clocks — pure domain logic, no database.

3. The contributor seam — teardown that scales with features

Now hole two. The platform's answer, in full:

```
// src/Core/Abstractions/ITenantDataContributor.cs — the real file
/// <summary>
/// A feature's contribution to tenant-data presence and teardown. Each vertical slice
/// that owns tenant-scoped tables registers one of these in DI; the platform queries
/// all contributors instead of a hard-coded method — so adding a feature never means
/// editing a central "has data" / "wipe data" method.
/// </summary>
public interface ITenantDataContributor
{
    /// <summary>True if this contributor holds any data for the tenant.</summary>
    Task<bool> HasDataAsync(Guid tenantId, CancellationToken ct = default);

    /// <summary>Removes this contributor's data for the tenant. Called within the
    /// dissolve transaction, so it shares the caller's unit of work — do not commit
here.</summary>
    Task WipeAsync(Guid tenantId, CancellationToken ct = default);

    /// <summary>The section name this contributor's data appears under in a tenant
    /// export (GDPR-1), e.g. "notes". Stable and unique across contributors.</summary>
    string ExportKey { get; }

    /// <summary>A JSON-serializable snapshot for a data export. Null when nothing to
    /// include. Identifiers and content only — never secrets.</summary>
    Task<object?> ExportAsync(Guid tenantId, CancellationToken ct = default);
}
```

The inversion is the entire idea. The platform doesn't know what features exist; it asks DI for *all* `ITenantDataContributor`s and fans out. A new feature ships its own contributor (five minutes: "do I have rows for this tenant / delete them / here's my export"), registers it, and is *automatically* part of dissolve, of the join-time data check, and — Part 7 — of GDPR export. **The platform is closed to modification, open to extension**: Open/Closed not as a poster slogan but as a DI query.

Notice the interface already carries `ExportKey`/`ExportAsync`. Historically the seam was born two-method and GDPR *widened* it — and because widening an interface breaks every implementor at compile time, each existing feature was forced, by the build, to answer "what do I export?" That's the seam working even while changing: the compiler runs the migration checklist. (The course teaches the final four-member shape from the start; you get the anecdote instead of the breakage.)

The orchestrator stays platform-small:

```
// src/Api/Services/TenantDissolutionService.cs (the real shape)
/// Fan out over every ITenantDataContributor so each feature wipes its own domain
/// data first, then run the platform's core teardown (memberships, invitations, the
/// tenant row). Deliberately does NOT open its own transaction — callers invoke it
/// inside their existing dissolve transaction, so the whole thing commits atomically.
public async Task DissolveAsync(Guid tenantId, CancellationToken ct = default)
{
    foreach (var contributor in contributors) // injected:
IEnumerable<ITenantDataContributor>
        await contributor.WipeAsync(tenantId, ct);
    await tenantRepository.WipeDataAsync(tenantId, ct); // core: memberships, invites,
tenant row
}
```

Two disciplines in the comments deserve promotion to principles. **Order:** features first, core last — contributors may need the tenant's rows (FKs) to still exist while they wipe. **Transactionality:** the service *enlists* in the caller's transaction rather than owning one — dissolve is one atomic event alongside the re-home and audit steps that surround it (2.8's leave flow); a crash mid-wipe rolls back to "still dissolved-nothing," never "half-dissolved." A teardown that can partially complete is a data-integrity bug with a delay timer. And who *calls* dissolve? Only system-level flows — the sole-owner leave and (later) account erasure — running on the system context from 2.7; the contributors' `WipeAsync` implementations are exactly the sanctioned cross-tenant-write code that trust level exists for.

4. Harden — the seam that can't be forgotten, provably

The seam's Achilles heel: a feature that *doesn't* register a contributor silently opts out of teardown. Convention again — unless a test closes it. The reference repo's `WipeDataTests` does exactly that, with 1.3's discovery trick pointed at a new target: walk the EF model for `ITenantScoped` entity types, dissolve a seeded tenant, then assert **zero rows remain in any tenant-scoped table**:

```
[Fact]
public async Task Dissolve_leaves_no_tenant_scoped_rows_behind()
{
    var tenantId = await SeedEveryFeaturesSampleDataAsync(); // incl. YOUR new feature

    await dissolution.DissolveAsync(tenantId);

    foreach (var entityType in db.Model.GetEntityTypes()
                 .Where(t => typeof(ITenantScoped).IsAssignableFrom(t.ClrType)))
        Assert.Equal(0, await CountRowsForTenantAsync(entityType, tenantId));
}
```

A feature that adds a tenant-scoped table without wiring teardown turns this red with the *table's name in the failure*. Model-derived, so it covers tables that don't exist yet — the same move as the fixture reset (R11) and the filter-coverage invariant (R2), now guarding deletion. Three guards, one pattern: **derive the checklist from the model; never hand-maintain it.**

5. Red — the tests

```
Scenario: invitation joins the redeemer's real account
  Given Ana invited ben@work.test
  And Ben signs in as ben@personal.test and presents the token
  Then Ben's account joins Ana's household as member
  And Ben's empty solo tenant is dissolved

Scenario: joining with data requires explicit consent to lose it
  Given Ben's solo household contains a Note
  When Ben redeems Ana's invite without confirmation
  Then the join is refused with has_data
  And nothing changed

Scenario: dissolve is total and atomic
  Given a tenant with data in every feature
  When the sole owner leaves with confirmation
  Then no tenant-scoped rows remain for that tenant
  And a wipe failure in any contributor rolls the entire dissolve back

Scenario: invitations are single-use and revocable
  Given a pending invitation
  When it is redeemed / revoked / expires
  Then subsequent redemption attempts fail with the same generic error
```

The third scenario's rollback half is worth the extra test plumbing (a contributor rigged to throw): atomicity claims are exactly the claims that rot silently.

6. Run it

```
dotnet test tests/Api.Tests --filter "Invitation|WipeData"
# live: invite yourself at a second address; open Mailpit; redeem on /join while
# signed in as another test account; GET /api/household → two members.
```

7. Architecture Decision

ARCHITECTURE DECISION

The fork: how does per-feature data participate in tenant teardown (and later, export)? (a) A central `WipeTenant()` that each feature edits; (b) database cascade deletes from the tenant row; (c) a DI-discovered contributor seam + a model-derived completeness test.

Chosen: (c) (ADR-003 lineage; ADR-011 widens it). Fan-out inverts the dependency: the platform stops needing to know its features. The completeness test converts the seam's one weakness (forgetting to register) into a named build failure. The same seam answers three lifecycle questions — has-data (join guard), wipe (dissolve), export (GDPR) — so a feature declares its data relationship *once*.

Rejected: (a) — the forget-to-edit design; its failure mode is silent orphaned data with legal implications. (b) — cascades handle *rows* but not the rest (files on disk in Part 6, provider-side subscriptions in Part 5 — dissolve is not only SQL), offer no has-data or export answer, and hide teardown semantics in migration DDL where no test names them.

The trade: every feature carries ~30 lines of contributor boilerplate, and dissolve is O(features) sequential queries — perfectly acceptable for an event as rare as tenant death, and the boilerplate is the checklist (3.2's slice template includes it pre-written).

8. Checkpoint

```
git add -A && git commit -m "feat: invitations + dissolve — contributor seam, atomic
teardown, completeness gate"
git tag lesson-2.9
```

Part 2 complete. The chassis stands: identity (tokens, passwordless, OAuth), tenancy (read filter, write stamping, EnterTenant), the tenant model with its invariants, and lifecycle machinery that future features join by construction. Every lesson from here *rides* this.

You should now be able to: separate "who holds the invitation" from "who is joining" and say why forwarding is safe; implement the contributor seam and its completeness test from memory; explain wipe ordering and transaction enlistment; and recognize Open/Closed when it's a DI query rather than a slogan.

Next: *Part 3 — The slice pattern & the UI.* The chassis gets its assembly manual: the repository seam, the anatomy of a vertical slice, and the first pixels.